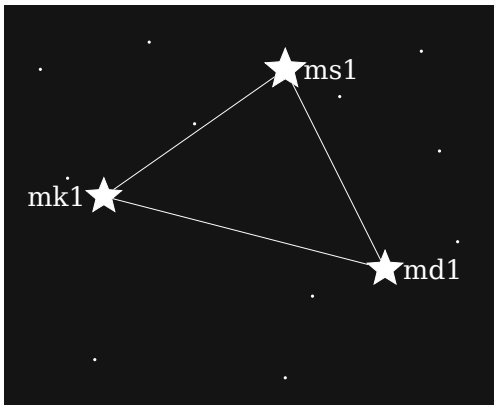
The m-format Constellation User Manual

Steel-engravable Bitcoin self-custody backup with `mnemonic-toolkit`, `md-cli`, `ms-cli`, and `mk-codec`

Brian Goss, MD

May 8, 2026



Contents

Chapter 1

Read this first — UNTESTED ALPHA SOFTWARE

This software has not yet been independently tested, audited, or proven in production. The m-format constellation is alpha-quality work-in-progress: the four codecs, the four CLIs, the BCH error-correcting math, the BIP-388 wallet-policy emission path, and the cross-card binding invariants have all been authored and reviewed only by the original developer.

Do not use this software to back up significant sums of money at this time. Doing so is tantamount to asking to be rekt.

1.1 Acceptable uses today

- Disposable amounts only (a few sats, on mainnet or testnet).
- Evaluation, learning, code review.
- Reproducing the published worked-example transcripts.
- Integration smoke-testing.

1.2 Unacceptable uses today

- Production multisig wallets covering meaningful balances.
- Inheritance plans, legacy-estate setups.
- Any wallet you would be unhappy to lose entirely.

1.3 Why this page is here

The format itself — the wire encoding, the BCH math, the BIP-388 mapping, the cross-card cryptographic binding — is intended to mature through external review and independent implementation. Reference implementations carry no warranty and no guarantee of correctness, ever — but the situation is especially acute today, before

any independent audit has happened. When the project reaches a stable release with documented external review, this page will be replaced with a stability-claim page.

Until then: assume bugs. Read the source. Verify your engraving with your own re-implementation if it matters.

Chapter 2

About this manual

The m-format constellation is a family of four sibling Bitcoin self-custody backup formats that engrave together as a coherent steel-engravable bundle. This manual is the end-user companion to the four formats and to the `mnemonic-toolkit` integration tool.

> **MIT License.** This manual is freely redistributable under MIT terms. See the `LICENSE` file in the source repository.

This is **manual v0.1**, tracking `mnemonic-toolkit main` (initial sync targets toolkit `v0.8.0`). The manual is a *living document*: each toolkit tag triggers a fresh PDF build attached to the corresponding GitHub release.

2.1 Who is this manual for?

The manual is **two-track**:

- **Bitcoin power users** — readers familiar with BIP-39 mnemonic phrases, BIP-32 derivation paths, descriptors, and multisig — read the main flow of every chapter and skip the `:::primer` boxes.
- **Newcomers** — readers who know what a Bitcoin wallet is but have not engraved a steel backup, do not yet know what BIP-388 is, or have never set up multisig — read the `:::primer` boxes for the background a chapter assumes, and consult Part VI's deep-dive appendices for full primers.

If you do not yet know what BIP-39, BIP-32, descriptors, or multisig *are*, start with Appendix B (BIP-39), Appendix C (BIP-32), and Appendix D (descriptors / BIP-388). Then come back to the front and read forward.

2.2 How to read this manual

The chapters are divided into six parts:

Part	Chapters	Read order
I — Foundations	Welcome, navigation, concept signposts	Read first.
II — Quick start	Install, first bundle, verify, recover	Read next; gives a working bundle in under 30 minutes.
III — Guided workflows	8 end-to-end recipes (single-sig, multisig, taproot multi, watch-only, recovery, migration, wallet export, BIP-85 children)	Skim the table of contents and read the workflows that match what you're building.
IV — CLI reference	`mnemonic`, `md`, `ms`, `mk`	Reference; consult per command.
V — Comparing & contrasting	7 decision-oriented chapters	Read to choose between overlapping features.
VI — Appendices	Glossary + 4 newcomer primers + test seeds + troubleshooting + release history + index	Mix of reference and primer material.

Every example in this manual uses the canonical BIP-39 test vector `abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about`. This phrase is **public** and any wallet derived from it has been swept by chain watchers. **Never engrave it. Never fund it.** Each example chapter opens with a `:::danger` admonition restating this.

2.3 Versioning and releases

The manual is a *living document*. Each `mnemonic-toolkit` GitHub release attaches a freshly-built PDF asset (`m-format-manual-toolkit-${TOOLKIT_VERSION}.pdf`). A breaking content change to the manual itself bumps `manual-MAJOR`; new chapters bump `manual-MINOR`; copyedits and typo fixes bump `manual-PATCH`. Independent of toolkit semver.

Chapter 3

Welcome to the m-format constellation

Self-custodying Bitcoin starts with one secret — your seed phrase — and an inevitable question: *how do I keep that intact for decades?* The m-format constellation is a family of four sibling formats that answer the question by splitting the backup into independently-checksummed, steel-engravable cards.

3.1 The four siblings

Card	Carries	What it answers
ms1	BIP-39 entropy	"What were the seed words?"
mk1	xpub + origin (master fingerprint + BIP path)	"What public key did the wallet use, and at which derivation?"
md1	wallet policy (template + bound xpub)	"What was the wallet's *spending rule* — single-sig? 2-of-3 multisig? taproot?"

A fourth piece — the **`mnemonic-toolkit`** CLI — is the *integration layer* that synthesises a coherent bundle from your inputs and verifies the cross-bindings. It does not engrave its own card; it produces the three card strings (ms1, mk1, md1) plus the visual engraving cards.

3.2 Why three cards instead of one

A single seed phrase, engraved on steel, recovers a wallet *if and only if* you also remember the wallet's spending rule. For a single- sig BIP-84 wallet, that's implicit:

most wallet apps will guess. For a multisig wallet — a 2-of-3 with two cosigners' xpubs and a taproot internal key — the seed alone is insufficient. You need:

- the **secret** (ms1) to sign,
- the **public keys** (mk1 cards from each cosigner) to construct the multisig script,
- the **policy** (md1) to know *what kind* of multisig.

Engraving all three on independent steel cards lets each card be stored separately and hardened against different threats. The redundancy is **asymmetric**: a surviving `ms1` plus knowledge of the wallet template suffices to re-derive `mk1` and `md1`, but losing all copies of `ms1` means losing spending capability permanently — the public cards alone reconstruct only a watch-only setup. Each card also carries its own BCH error-correction checksum, so per-card mis-stamps and minor surface damage are locatable, not catastrophic (Appendix E gives the precise per-card detection / correction guarantees). Multisig adds threshold-level redundancy across cosigners' `ms1` cards: in a 2-of-3 wallet, one cosigner's `ms1` may be lost without losing spending capability; two cannot. Chapter 35 walks through the full recovery matrix.

> **Cross-binding, in one sentence.** Each mk1 card carries a 4-byte `policy_id_stub` — `SHA-256(canonical wallet-policy preimage)[0..4]` — that is also computable from each md1 card. If you mix cards from different wallets, the stubs disagree and `mnemonic verify-bundle` fails fast.

3.3 How to use this manual

Most readers want one of three things:

1. **"Show me what to do, fast."** Read [Part II — Quick start] (#your-first-bundle). 30 minutes, ends with a working bundle on paper.
2. **"I have this *specific* situation: 2-of-3 multisig with three cosigners on three machines."** Read Part III — Guided workflows. Eight end-to-end recipes covering single-sig, multisig, taproot, watch-only, recovery, migration, wallet export, and BIP-85 children.
3. **"I want to know how this compares to SLIP-39 / Shamir / naked BIP-39."** Read [Part V — Comparing & contrasting] (#m-format constellation-vs-slip-39-vs-naked-bip-39-vs-shamir).

The CLI reference (Part IV) and the appendices (Part VI) are consulted, not read top-to-bottom.

3.4 Status

This manual is **v0.1**, tracking `mnemonic-toolkit main` (initial sync targets toolkit `v0.8.0`). Each new toolkit tag triggers a fresh PDF build attached to the corresponding GitHub release. The release-history appendix summarises what each toolkit version added or broke; the toolkit's own `CHANGELOG.md` is the authoritative source.

Chapter 4

How to read this manual

You don’t need to read straight through. The structure is designed so power users can skim, newcomers can fill in gaps from the appendices, and reference users can jump to the index.

4.1 The two-track design

Two kinds of reader, one document. Most chapters address Bitcoin power users by default — the prose assumes you already know what a BIP-39 phrase is, why a wallet has a derivation path, and what “multisig” means at the script level. When a chapter would otherwise need to detour into background, it does so inside a fenced `:::primer` box like this:

> A short paragraph (≤80 words) explaining just enough background for the chapter’s main flow. Power users skip these. The deeper version is in the appendix the box links to.

If you find the primer boxes useful, you’re a newcomer. Read them all and supplement with the matching appendix when a topic clicks incompletely. If you find them noise, ignore them and read the main flow.

4.2 A separate kind of box for warnings

When a chapter introduces or first uses a value that is *publicly known* and therefore unsafe to engrave, it opens with a `:::danger` admonition. Throughout this manual, every example uses the canonical BIP-39 test vector `abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about` — which is **public** and has been swept by chain watchers. Each example chapter restates that warning on its first use.

If you copy an example *literally*, you will derive addresses that have already been emptied. **Never engrave the canonical test seed. Never fund it.** Generate fresh entropy for your real wallets.

4.3 Six parts, six purposes

Part	Purpose	Read when
I — Foundations	Establish the four-card mental model and this manual's conventions.	First.
II — Quick start	Install the toolkit; produce and verify your first bundle.	After Part I, before anything else.
III — Guided workflows	Eight end-to-end recipes: single-sig, multisig, taproot, watch-only, recovery, migration, wallet export, BIP-85 children.	Pick the one that matches what you're building.
IV — CLI reference	Per-binary, per-subcommand flag reference for `mnemonic`, `md`, `ms`, plus a Rust-API tour of `mk-codec`.	Consult by command, not by reading.
V — Comparing & contrasting	Decision-oriented chapters: when to use which format, which descriptor variant, which passphrase channel.	When two features look similar and you need to choose.
VI — Appendices	Glossary; newcomer primers for BIP-39 / BIP-32 / descriptors / codex32; test-seed handling; troubleshooting; release history; index.	Reference.

4.4 Cross-references

Every chapter links forward and backward. Internal links point to the target's section anchor; the markdown render resolves these inside the concatenated `m-format-manual.md`, and the PDF render resolves them as hyperlinked TOC entries.

The PDF includes a true page-numbered alphabetical index built from inline `\index{}` markers throughout the source. The markdown render strips those markers; instead, Appendix I ships a curated `Term → §section` table that is held in lockstep with the LaTeX index by the lint script.

If you write contributions back to this manual. See `docs/manual/AUTHORING.md` in the toolkit repository. It documents the convention for `:::primer` / `:::danger` boxes, inline `\index{}` placement and its mirror requirement on Appendix I, the

canonical-test-seed DANGER policy, glossary discipline, and the pre-commit lint expectations.

Chapter 5

Concept signposts

The rest of this manual assumes a working knowledge of four Bitcoin concepts. This chapter is a one-paragraph signpost for each, with a link to the full primer in Part VI.

5.1 BIP-39 — wallet entropy as words

BIP-39 turns a randomly-generated chunk of entropy (typically 128 or 256 bits) into a sequence of 12 or 24 words drawn from a fixed 2048-word list. The words encode the entropy losslessly; the same words always recover the same wallet. The `ms1` card carries this entropy in a checksum-protected, steel-engravable form. *Deep dive:* Appendix B.

5.2 BIP-32 — a tree of keys from one seed

A BIP-39 phrase deterministically yields a single master extended private key. From that root, BIP-32 derives a tree of child keys addressable by *paths* like `m/84'/0'/0'/0/0`. Wallets pick a *purpose* path (BIP-44, BIP-49, BIP-84, BIP-86) and an *account* index, then derive change and address keys from there. The `mk1` card carries the *origin* — master fingerprint and the path used — plus the resulting xpub. *Deep dive:* Appendix C.

5.3 Descriptors — the wallet's spending rule

A descriptor is a small string that tells Bitcoin Core (and other descriptor-aware wallets) *exactly* how to construct a wallet's addresses, including the script type (legacy / SegWit / taproot), the keys involved, the multisig threshold, and the derivation. A single-sig P2WPKH wallet's descriptor is simple: `wpkh(xpub.../0/*)`. A 2-of-3 multisig descriptor is `wsh(sortedmulti(2, xpub_a/0/*, xpub_b/0/*, xpub_c/0/*))`. The **`md1`** card carries the descriptor as a wallet *policy* (BIP-388 template + bound keys). *Deep dive:* Appendix D.

5.4 Multisig — K of N must sign

A multisig wallet requires *at least K* out of N cosigners to produce a valid signature. K is set via `--threshold K`; any value 1..=N is valid. Multisig changes neither the seed (each cosigner has their own) nor the address derivation — it composes them through a script. The toolkit takes each cosigner via `--slot @N.<subkey>=<value>`; the descriptor card encodes the resulting multisig policy. *Deep dive for newcomers:* Appendix D covers descriptors broadly; multisig-specific worked examples are in the multisig workflow.

5.5 Codex32 / BCH error correction

Each card's last few characters are a checksum derived from a Bose–Chaudhuri–Hocquenghem (BCH) polynomial. The `ms1` card uses BIP-93 codex32 directly; the `mk1` and `md1` cards use a BCH variant that mixes the human-readable prefix into the polynomial — same algebra, different polynomial. A handful of bit errors in the engraving are detected and *located*, so a partially-damaged card can usually be corrected. *Deep dive:* Appendix E.

You can now read forward into Part II — Quick start and produce your first bundle. Or jump to whatever's relevant to your situation.

Chapter 6

Installing the toolkit

The m-format constellation ships as four crates across four sibling repositories, each with a standalone CLI binary (`mnemonic`, `md`, `ms`, `mk`). This chapter installs the four binaries.

> **The four sibling repos** are `bg002h/mnemonic-toolkit` (CLI: `mnemonic`), `bg002h/descriptor-mnemonic` (CLI: `md`), `bg002h/mnemonic-secret` (CLI: `ms`), and `bg002h/mnemonic-key` (library `mk-codec` plus CLI `mk`, since v0.2). They are published to crates.io as soon as their dependencies land there; until then, install from source via `cargo install --git`. Future versions will pin to crates.io directly.

6.1 Pre-requisites

You need a recent **Rust toolchain** — the `rust-toolchain.toml` in each repository pins `1.77+`. Install via `rustup` if you do not have it:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

(Or use your distribution's package manager. Verify with `cargo --version` and `rustc --version`.)

6.2 Path A — install from source via cargo

This is the recommended path until crates.io publication completes:

```
cargo install --locked --git https://github.com/bg002h/mnemonic-toolkit.git
 ↪  mnemonic-toolkit
cargo install --locked --git https://github.com/bg002h/descriptor-mnemonic.git md-cli
cargo install --locked --git https://github.com/bg002h/mnemonic-secret.git ms-cli
cargo install --locked --git https://github.com/bg002h/mnemonic-key.git --tag
 ↪  mk-cli-v0.2.0 --bin mk
```

Each command compiles the source and writes the binary into `~/.cargo/bin/`. Make sure that directory is on your `PATH` (most rustup installations add it automatically).

Verify the binaries:

```
mnemonic --version
md --version
ms --version
mk --version
```

6.3 Path B — clone and build (for contributors)

If you intend to read the code, modify it, or run the test suites:

```
git clone https://github.com/bg002h/mnemonic-toolkit.git
git clone https://github.com/bg002h/descriptor-mnemonic.git
git clone https://github.com/bg002h/mnemonic-key.git
git clone https://github.com/bg002h/mnemonic-secret.git

cd mnemonic-toolkit && cargo build --release --bin mnemonic
cd ../descriptor-mnemonic && cargo build --release --bin md
cd ../mnemonic-secret && cargo build --release --bin ms
cd ../mnemonic-key && cargo build --release --bin mk
```

The release binaries land in each repo's `target/release/`. Either copy them onto your `PATH` (`cp target/release/mnemonic ~/.local/bin/`) or invoke them directly.

The four repos coordinate via cross-repo `FOLLOWUPS.md` mirrors and shared release-cycle conventions; clone them as siblings rather than nested. The `CLAUDE.md` files in each repo are non-binding guidance for AI-assisted contributions.

6.4 Path C — Docker (CI / reproducible builds)

For CI and reproducible installations, the toolkit ships a build image at `docs/manual/Dockerfile.b` (used by `make pdf-docker` for the manual). For the *binaries themselves* there is no distribution image yet — see the follow-up on cargo-publishing the workspace; that's the prerequisite for official binary releases.

6.5 Verifying your install

A trivial smoke check that all four CLIs respond:

```
mnemonic --help | head -5
md --help | head -5
ms --help | head -5
mk --help | head -5
```

You should see a short usage banner from each. Now read on to Your first bundle — a single-sig BIP-84 walkthrough that produces three real card strings on your terminal.

Chapter 7

Your first bundle

This chapter produces a complete 3-card bundle for a single-sig BIP-84 mainnet wallet. End-to-end takes about three minutes.

> Every command in this manual uses the canonical BIP-39 test vector `abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about`. This phrase is **public** — chain watchers have swept any wallet ever derived from it. **Never engrave or fund a wallet built from the canonical test seed.** Generate fresh entropy for real wallets (see Test seeds and example data).

7.1 Prerequisites

You should have run Path A or Path B of the previous chapter and verified `mnemonic --version` reports `0.8.0` or later.

7.2 The command

```
mnemonic bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about"
```

Three flags carry the work:

- **`--network mainnet`** — the wallet is for Bitcoin mainnet. Other values: `testnet`, `signet`, `regtest`.
- **`--template bip84`** — single-sig BIP-84 (native SegWit, P2WPKH, derivation `m/84'/0'/0'`). For multisig and taproot variants, see the Workflows part.
- **`--slot @0.phrase=…`** — cosigner index `@0` (only one for single- sig), `phrase` subkey (the BIP-39 mnemonic). For multisig wallets, you'd add `--slot @1.phrase=…`, `--slot @2.phrase=…`, etc.

7.3 Output

Three card strings, each shown twice — once contiguous (handy for copy-paste into a wallet) and once chunked (handy for engraving):

```
# ms1 (entropy, BCH-checksummed)
ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f

ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqqqq qqcj9 sxraq 34v7f

# mk1 (xpub + origin)

mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8q
mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh

mk1qp rsqhp qqsq3 cqtsl eeutk s2qvz g3vs7 0mejh k622w s2kgd
emj2c d8zwj 2skzx 2wq0q w70l4 q99vd yh5x0 z8v4y slsp8 qp3yx
g3dpe 854wq 4
mk1qp rsqhp p0f30 mtxzd 65mvw cur9u sdatw uqvq6 z70r9 nwrgk
6xn6l 8gy6n wa2n9 77sw6 zh34r ma0nh

# md1 (wallet policy)
md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0
md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc
md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6

md1zs-xdspq-qqpm6-jzzqq-vqz6q-u79mg-9p2sg-fff6p-2eph8-wftp5-uf6gq-nlgzq-qqnym-v0
md1zs-xdspq-259s3-jnsrc-rhnla-gpftr-f9apn-c3m9f-y8uqf-c85ch-a4nqn-h5k67-ey2hz-yc
md1zs-xdspq-jd65m-vwcur-9usda-twuqv-q6z70-r9nwr-gk6xn-6l8gy-6nvqh-uuyvz-gaeja-h6

# === Wallet bundle: bip84, mainnet ===
# ms1: 1c017
# mk1: 1c017
# fingerprint: 73c5da0a
# origin path: 84'/0'/0'
# Template: bip84
# md1: 1c01
warning: secret material on stdout — consider redirecting (e.g., '> file.txt' or '| age -e ...')
```

7.4 Reading the output

Three card sections, each with a header comment, contiguous string, chunked string, and a trailing `# === Wallet bundle:` block summarising the bundle's metadata.

For this single-sig wallet:

Card	Contiguous form	Use
ms1	`ms10entrsqq…34v7f`	Engrave on the *secret* card. Recovers the seed.

Card	Contiguous form	Use
mk1	`mk1qprsqhp…854wq4` (line 1) and `mk1qprsqhp…ma0nh` (line 2)	Engrave on the *key* card. Two strings because the xpub is too long to fit one BCH-checksummed group at the toolkit's chunking density.
md1	`md1zsxdspq…` (three lines)	Engrave on the *descriptor* card. Three strings encode the wallet policy and bind it to the xpub.

The last block is **not** part of the engraving — it's a bundle *summary* showing the version stamps (`1c017`), the master fingerprint, the origin path, the template, and the md1 version. Use it to visually compare two bundles.

The trailing `warning: secret material on stdout` is real and worth heeding for production: redirect the output to a file or pipe to `age -e` for encryption-at-rest before saving.

7.5 What about the engraving cards themselves?

The toolkit also emits an *engraving-card* layout to stderr by default; pass `--no-engraving-card` to suppress it. The card is plain text — suitable for verification before committing to physical engraving. See Single-sig steel-engraved backup workflow for the full ceremony, including which strings go on which physical plate, plus visual checksum verification.

7.6 Verifying the bundle

Run the verify-bundle walkthrough next — it re-derives the expected card content from the seed and confirms each card decodes and matches.

Chapter 8

Verifying a bundle

After producing a bundle (and ideally before engraving), confirm it round-trips. The `mnemonic verify-bundle` subcommand re-derives the expected card content from the seed and reports per-card pass/fail plus the overall result.

8.1 The command

Pass the original `--slot @0.phrase=…` plus the cards that came back from your bundle invocation. (Same canonical test seed as Chapter 22; see the DANGER box there.) For a single-sig BIP-84 mainnet wallet with one ms1, two mk1 strings, and three md1 strings:

```
mnemonic verify-bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --ms1 ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f \
  --mk1
   ↪  mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp
   ↪  \
  --mk1
   ↪  mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh
   ↪  \
  --md1 md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0 \
  --md1 md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc \
  --md1 md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6
```

The flag set:

- **`--ms1 <STRING>`** takes one ms1 card.
- **`--mk1 <STRING>`** can be repeated; for this BIP-84 wallet the bundle emits two mk1 strings, so you pass `--mk1` twice.
- **`--md1 <STRING>`** can be repeated; this wallet emits three md1 strings.

The same `--network`, `--template`, and `--slot` flags as the original `bundle` invocation are required so the verifier knows what *should* match.

8.2 Output

```
ms1_decode: ok decoded successfully
ms1_entropy_match: ok ms1 byte-identical
mk1_decode: ok decoded successfully
mk1_xpub_match: ok xpub matches
mk1_fingerprint_match: ok fingerprint matches
mk1_path_match: ok path matches
md1_decode: ok decoded successfully
md1_wallet_policy: ok wallet-policy mode confirmed
md1_xpub_match: ok 65-byte xpub matches expected
result: ok
```

Each line is a discrete check. The four checks per card type are:

Check	What it confirms
`ms1_decode` / `mk1_decode` / `md1_decode`	The card string parses back to a structured value (BCH checksum verified, internal layout matches the format).
`ms1_entropy_match`	The decoded entropy round-trips byte-for-byte from the supplied `--slot @0.phrase=…`.
`mk1_xpub_match`	The xpub on the mk1 card equals the xpub re-derived from the seed at the same origin.
`mk1_fingerprint_match`	The mk1 card's master fingerprint matches the seed's.
`mk1_path_match`	The mk1 card's origin path matches the template's expected derivation path.
`md1_wallet_policy`	The md1 card was emitted in wallet-policy mode (vs. legacy descriptor mode).
`md1_xpub_match`	The xpub bound in md1 matches the xpub re-derived from the seed.

Final line `result: ok` means the bundle is internally consistent and matches the seed. `result: mismatch` would mean one of the sub-checks failed; the failing line names the cause.

8.3 Why this matters before engraving

Verification protects against three failure modes:

1. **Transcription errors.** If you typed a card wrong from the bundle output, `*_decode` fails — the BCH checksum catches it.
2. **Wrong template / network.** If you bundled BIP-84 mainnet but typed `--template bip49` here, every `*_match` for the expected- xpub-side fails.

3. **Mixed-bundle confusion.** If you passed an mk1 card from a different wallet, `policy_id_stub` mismatches via the md1↔mk1 cross-binding (covered in the recovery-paths workflow).

Run `verify-bundle` once after every bundle synthesis. If it returns `result: ok`, you are safe to engrave.

Chapter 9

Minimal recovery walkthrough

Recovery is the inverse of your first bundle: given engraved cards, produce a wallet usable for signing. This chapter walks the simplest case — single-sig, all three cards in hand. The recovery-paths workflow covers damaged or partial bundles.

9.1 Goal

Recover the seed phrase and a watch-only descriptor from the cards in your hand:

- ms1: `ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f`
- mk1: two strings (xpub + origin)
- md1: three strings (wallet policy + bound xpub)

The seed phrase signs spends. The descriptor (md1, decoded) tells a watch-only wallet — Bitcoin Core, Sparrow, Specter — how to construct addresses.

9.2 Step 1 — recover the seed phrase from ms1

```
mnemonic convert \
  --from ms1=ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f \
  --to phrase
```

Output:

```
phrase: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
 ↪  abandon about
warning: secret material on stdout — consider redirecting (e.g., '> file.txt' or '| age
 ↪  -e ...')
```

The phrase is the canonical BIP-39 12-word mnemonic. Your wallet of choice imports BIP-39 phrases directly.

> **Why ms1 instead of just engraving the phrase?** A 12-word phrase on steel is fragile — a single mis-stamped letter can turn a valid word into a different valid word, silently corrupting the seed. The ms1 encoding wraps the entropy in a BCH error-correction checksum so a small number of bit errors in the engraving are

detected and located. The phrase you'd write from memory is recoverable; ms1 is recoverable even with a few stamping mistakes.

9.3 Step 2 — confirm the public-key side via mk1

If you only need to *watch* the wallet (no signing), the seed isn't required. Decode the mk1 card via `mnemonic convert` to recover the xpub, master fingerprint, and origin path:

```
mnemonic convert \
  --from mk1=␣
   ↪  "mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4ysls
   ↪  mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh"
   ↪  \
  --to xpub --to fingerprint --to path
```

Output:

```
xpub:
 ↪  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVU
fingerprint: 73c5da0a
path: 84'/0'/0'
```

The two mk1 strings are passed as a single space-separated value; the toolkit reassembles them and verifies the BCH checksum on each. Alternatively, `mk decode <mk1-string-1> <mk1-string-2>` recovers the same fields directly from the standalone `mk-cli` binary without bundling the entire toolkit (useful on minimal-surface recovery machines).

9.4 Step 3 — re-derive the descriptor from md1

The md1 card carries the wallet policy. Decoding it tells your watch-only wallet what kind of multisig (or single-sig) script to expect. Pass the strings positionally to `md decode`:

```
md decode \
  md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0 \
  md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc \
  md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6
```

Output:

```
wpkh(@0/<0;1>/*)
```

For this BIP-84 single-sig bundle, the decoded wallet-policy template is `wpkh(@0/<0;1>/*)` — single key, native segwit, with `<0;1>` for external-and-change descriptor pairs (BIP-388 multipath).

9.5 Step 4 — verify before trusting

Before importing the seed into a hot wallet, run the verify-bundle walkthrough once more — this time using the values you just decoded — to confirm the cards agree with each other across the cross-binding (`policy_id_stub`).

If `result: ok`, the bundle is intact. Import the seed into your signing wallet of choice; import the descriptor into your watch-only wallet of choice. The wallet-export workflow covers Bitcoin Core / BIP-388 / Sparrow / Specter export shapes in detail.

That's the minimal recovery — three cards in, one phrase and one descriptor out. From here:

- For ceremony details (which physical plate carries which string, visual-checksum verification before importing), read Single-sig steel-engraved backup workflow.
- For damaged-card scenarios (lost ms1, partially-stamped mk1, illegible md1), read Recovery paths by damaged-card scenario.

Chapter 10

Single-sig steel-engraved backup

The full ceremony for a single-cosigner BIP-84 mainnet wallet, from fresh entropy to three engraved plates. Allow about 90 minutes; most of that is the engraving itself.

This chapter assumes you have read Your first bundle and Verifying a bundle. The CLI mechanics are identical here; what is added is the *physical* discipline.

> Examples in this chapter use the canonical BIP-39 test vector `abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about`. This phrase is **public** — chain watchers have swept any wallet ever derived from it. **Never engrave or fund a wallet built from the canonical test seed.** For real bundles, generate fresh entropy on an air-gapped device.

10.1 Ceremony at a glance

10.2 Step 1 — generate fresh entropy off-line

Use any reputable BIP-39 generator: a hardware wallet's "new seed" flow, the Coldcard's dice roller, or an offline machine running `bitcoinjs` in a browser kept off the network. The toolkit accepts any standards-conforming 12 / 15 / 18 / 21 / 24-word mnemonic in any of the ten BIP-39 wordlists.

Write the resulting phrase down on scratch paper *only for the duration of the ceremony*. The phrase will live on the ms1 card and in your head; the scratch paper is destroyed at the end.

10.3 Step 2 — synthesise the bundle

```
mnemonic bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --self-check \
  > bundle.txt
```

Three additions vs. Your first bundle:

- **`--self-check`** — the toolkit re-parses its own emitted bundle and verifies it round-trips before exiting. A bug in card synthesis surfaces here, not later when you have stamped two plates and discover the third disagrees.
- **redirect to `bundle.txt`** — keeps secret material off the scrollback buffer.
- **omit `--no-engraving-card`** (the default behaviour) — the engraving-card layout is emitted on stderr, separate from the copy-pasteable card strings on stdout. The layout is the visual aid you align against when stamping.

10.4 Step 3 — verify before stamping

`bundle.txt` now contains the three card strings. Read them back with `mnemonic verify-bundle`:

```
mnemonic verify-bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --ms1 ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f \
  --mk1
   ↪  mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp
   ↪  \
  --mk1
   ↪  mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh
   ↪  \
  --md1 md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0 \
  --md1 md1zsxdspq259s3jnsrcrhnlagpftrf9apnc3m9fy8uqfc85cha4nqnh5k67ey2hzyc \
  --md1 md1zsxdspqjd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nvqhuuyvzgaejah6
```

The `result: ok` final line is the green light to begin stamping.

10.5 Step 4 — stamp the three plates

The convention is colour-keyed. The `mnemonic-toolkit` engraving-card emission uses red for ms1, blue for mk1, green for md1; the same colours apply to physical plates if you label them.

Plate	Card	Strings to stamp	What it carries
Red	ms1	1	BIP-39 entropy
Blue	mk1	2 (concatenated xpub + origin)	xpub fingerprint + path
Green	md1	3 (concatenated wallet-policy chunks)	template + bound xpub

The chunked form (`ms10e ntrsq qqqqq qqqqq qqqqq qqqqq qqqqq qqcj9 sxraq 34v7f`) is the engraving guide — five-character groups separated by spaces align with the printed engraving cards.

Each plate carries its own BCH error-correction checksum. If you mis-stamp a single character (`s` instead of `5`, say), the codec located it during decode and tells you the *position* of the error so you can verify against the original. The single-string ms1 card tolerates more errors than the longer mk1/md1 strings, but all three are recoverable from a small number of stamping mistakes.

10.6 Step 5 — verify each engraved plate

Crucially: don't trust your own typing. After stamping, re-decode each plate by reading the engraved characters and running them through `mnemonic verify-bundle` again — *with the same flags as Step 3*. If `result: ok`, the plate is faithful to the bundle. If a sub-check fails, the diagnostic names the failing card (e.g. `mk1_decode: error at position 47`) and you know which character to inspect.

Use a magnifier; eyes lie about `0` vs `o` and `1` vs `l`. The codec alphabet is *deliberately* chosen to exclude visually-similar characters, but stamping artefacts (a struck depth difference, a misaligned letter) can mimic the wrong character at glance.

10.7 Step 6 — geographic separation

For meaningful protection against fire, flood, theft, or seizure, the three plates should rest in three independent locations. A common arrangement:

- **Red (ms1)** — primary safe, on-site, fireproof.
- **Blue (mk1)** — bank safe-deposit box.
- **Green (md1)** — trusted family member, attorney, or off-site vault.

The cross-binding via `policy_id_stub` means an attacker who steals only one plate cannot derive the wallet alone:

- ms1 alone reconstitutes the seed but not the wallet's spending rule (which derivation? which template?). Software that imports BIP-39 phrases will *guess*, often wrongly for non-default templates.
- mk1 + md1 alone is a watch-only bundle; spends are impossible without the seed.
- Any two plates lacking the third leave the wallet unrecoverable (or at least requires Bitcoin-forensics-level reconstruction).

The recovery side of this arithmetic is covered in Recovery paths by damaged-card scenario.

10.8 Variants

- **BIP-86 taproot single-sig.** Replace `--template bip84` with `--template bip86`. The derivation switches to `m/86'/0'/0'`; the ceremony is otherwise identical.
- **BIP-44 / BIP-49.** Same pattern, derivation changes to `m/44'/0'/0'` (legacy) or `m/49'/0'/0'` (nested-segwit).

- **Testnet / signet.** Replace `--network mainnet` with `--network testnet` or `--network signet`. The ms1 card re-encodes the same entropy, but the mk1's xpub serialises with the testnet prefix (`tpub`).
- **Privacy-preserving fingerprint.** Add `--privacy-preserving` to suppress the master fingerprint from the mk1 emission. Recovery software then re-derives the fingerprint from the seed at import.

For multisig variants — the more security-meaningful use of the m-format constellation — read the next chapter, Multi-source 2-of-3 multisig.

Chapter 11

Multi-source 2-of-3 multisig

The headline use of the m-format constellation: a 2-of-3 segwit multisig wallet whose three cosigners each generate their seed *on their own machine*. Each cosigner ends up with a personal ms1 card plus the same shared mk1-set + md1-set.

This chapter walks the *coordinated* synthesis flow first, where one laptop has all three phrases briefly during the bundle pass. For the fully air-gapped variant — where no machine ever sees more than one cosigner's secret — see the section "Air-gapped variant" at the end.

> Examples below use the canonical BIP-39 test vector `abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about` for cosigner 0 and two derived BIP-39 test vectors for cosigners 1 and 2. All three are **public**. Never engrave or fund a wallet built from these seeds.

11.1 What you build

Three cosigners produce **seven cards total**: three ms1 cards (one per cosigner), three mk1 cards (one per cosigner), and one md1 card (the shared wallet policy). Each cosigner's private engraving set is their own ms1 + the three mk1s + the one md1. The xpubs are public; sharing mk1s across cosigners is safe.

11.2 Step 1 — pick your template

`wsh-sortedmulti` is the segwit BIP-67 sorted-multisig template. The toolkit also offers `wsh-multi` (unsorted), `sh-wsh-sortedmulti` and `sh-wsh-multi` (nested-segwit, BIP-49-style addresses), and the two taproot variants `tr-multi-a` and `tr-sortedmulti-a`. For new wallets, prefer `wsh-sortedmulti` over `wsh-multi`: BIP-67 sorting means the cosigner order on the engraved md1 is irrelevant during recovery, so losing track of "who was @0?" doesn't brick the wallet.

For taproot multisig, see Taproot multisig.

11.3 Step 2 — synthesise the bundle

```
mnemonic bundle \
  --network mainnet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --slot @1.phrase="legal winner thank year wave sausage worth useful legal winner
   ↪  thank yellow" \
  --slot @2.phrase="letter advice cage absurd amount doctor acoustic avoid letter
   ↪  advice cage above" \
  --self-check \
  > bundle.txt
```

The interesting flags vs. single-sig:

- **`--template wsh-sortedmulti`** — BIP-67-sorted segwit multisig. The emitted descriptor will be `wsh(sortedmulti(2,@0,@1,@2))`.
- **`--threshold 2`** — the K of K-of-N. Required for multisig templates; ignored for single-sig.
- **`--slot @N.phrase=…`** repeated three times — one slot per cosigner, indexed `@0` through `@2`. The slot index becomes the cosigner index in the BIP-388 wallet policy.
- **`--multisig-path-family bip87`** is the default and stays. The toolkit derives at `m/87'/0'/0'` for each cosigner. To use the Coldcard / SeedSigner-friendly `m/48'/0'/0'/2'` BIP-48 family, pass `--multisig-path-family bip48`.

The output is the full seven-card bundle. Each cosigner's ms1 appears under a `# === Cosigner N ===` separator on the engraving-card stderr stream; the stdout strings are grouped per card type.

11.4 Step 3 — verify before stamping

```
mnemonic verify-bundle \
  --network mainnet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --slot @1.phrase="legal winner thank year wave sausage worth useful legal winner
   ↪  thank yellow" \
  --slot @2.phrase="letter advice cage absurd amount doctor acoustic avoid letter
   ↪  advice cage above" \
  --ms1 <ms1-cosigner-0> \
  --ms1 <ms1-cosigner-1> \
  --ms1 <ms1-cosigner-2> \
  --mk1 <mk1-cosigner-0-line-1> --mk1 <mk1-cosigner-0-line-2> \
  --mk1 <mk1-cosigner-1-line-1> --mk1 <mk1-cosigner-1-line-2> \
  --mk1 <mk1-cosigner-2-line-1> --mk1 <mk1-cosigner-2-line-2> \
  --md1 <md1-line-1> --md1 <md1-line-2> --md1 <md1-line-3> --md1 <md1-line-4>
```

The order of `--mk1` repetitions is a BIP-67 sorted set; the toolkit re-sorts on its side, so the on-screen ordering you pass does not have to match the canonical BIP-67 order. The verifier reports per-cosigner `mk1_xpub_match`, plus the across-cosigner

`md1_xpub_match` confirming each cosigner's xpub was bound into the shared md1 in the right position.

`result: ok` means all three personal sets check out and the shared md1 binds the right xpubs at the right cosigner positions.

11.5 Step 4 — stamp each cosigner's set

Each cosigner physically receives three plates — their own ms1, the three mk1s as a public xpub set, and the one shared md1.

A common arrangement:

Plate	Held by	Engraved with
Red (ms1)	only cosigner N	their own ms1 (1 string)
Blue×3 (mk1s)	every cosigner	three mk1 cards (2 strings each)
Green (md1)	every cosigner	shared md1 (3–4 strings)

The mk1 plates are public, so distributing them is harmless. The md1 plate is also public (it carries no secret). Only the personal ms1 plate must be guarded.

Stamping discipline (plate-by-plate, decode-after-each) follows the same drill as the single-sig ceremony; do not skip the post-stamp `verify-bundle` re-decode step.

11.6 Recovery dynamics

A 2-of-3 wallet is *spendable* when any two cosigners cooperate. It is *recoverable* with even fewer cards intact, depending on what remains:

Lost / damaged	Recovery
One cosigner's ms1	Other two cosigners' ms1s + the public mk1 set + md1 reconstruct the wallet. The lost ms1 is a regenerable surface (re-stamp from the still-valid phrase, if the cosigner can restate it).
One mk1 plate	Re-derive from any cosigner's seed via `mnemonic convert --from phrase=… --to mk1`. The mk1 carries no secret; rebuilding it is fast.
The md1 plate	Re-derive from `mnemonic export-wallet --template wsh-sortedmulti --threshold 2 --slot @N.xpub=…` (one slot per cosigner; only their xpubs are needed, no seeds).
Any *one* full card category (all ms1s, or all mk1s, or the md1)	Recoverable — the seeds reconstruct everything from scratch given the policy.

Lost / damaged	Recovery
Two ms1s + the md1	The threshold is 2; the surviving two seeds spend the wallet.
Three ms1s lost	Wallet is bricked — funds unrecoverable.

The detailed scenarios are in Recovery paths by damaged-card scenario.

11.7 Air-gapped variant

The above flow has all three phrases on one machine briefly during the `bundle` invocation. To keep each cosigner's seed strictly on their own air-gapped device:

1. **On each cosigner's machine**, run:

   ```
   mnemonic convert \
     --from phrase="<their phrase>" \
     --to xpub \
     --template wsh-sortedmulti \
     --account 0
   ```

   Output is the cosigner's xpub at the multisig derivation path.

2. **On the coordinator's machine**, with only the three xpubs (no secrets), run:

   ```
   mnemonic bundle \
     --network mainnet \
     --template wsh-sortedmulti \
     --threshold 2 \
     --slot @0.xpub=<xpub-0> \
     --slot @1.xpub=<xpub-1> \
     --slot @2.xpub=<xpub-2>
   ```

   This emits the *watch-only* bundle: three mk1s + one md1, no ms1. The watch-only bundle is what every cosigner engraves on their blue and green plates.

3. **On each cosigner's machine**, separately, run:

   ```
   mnemonic convert \
     --from phrase="<their phrase>" \
     --to ms1
   ```

   Output is just their personal ms1 string. Each cosigner engraves their own red plate without the coordinator ever seeing the seed.

The `policy_id_stub` cross-binding still holds: each cosigner's mk1 carries the stub computed from the shared wallet policy; the md1 re-computes the same stub. Tampering at the coordinator (e.g. swapping in an attacker's xpub for cosigner 2) makes the stubs disagree on cosigner 2's mk1 vs. md1, and `verify-bundle` flags it.

11.8 Variants

- **3-of-5, 4-of-7, etc.** — pass `--threshold K` and `--slot @N.…` for `N=0..N-1`. The toolkit caps at `1 ≤ K ≤ N ≤ 16` (BIP-388 limit).

- **`wsh-multi` (unsorted)** — replace `wsh-sortedmulti` with `wsh-multi`. The cosigner order on the engraved md1 then matters during recovery; track it explicitly.
- **Nested-segwit (`sh-wsh-sortedmulti`)** — BIP-49-style P2SH-wrapped segwit. Useful for legacy-only wallet compatibility; addresses start with `3…` rather than `bc1q…`.
- **Taproot (`tr-multi-a` / `tr-sortedmulti-a`)** — see Taproot multisig.
- **BIP-48 path family** — add `--multisig-path-family bip48` for `m/48'/0'/0'/2'` derivation (Coldcard, SeedSigner, Sparrow's default for native segwit multisig).

Chapter 12

Taproot multisig

Taproot multisig (`tr-multi-a` / `tr-sortedmulti-a`) puts the cooperative-spend script in a **script-path leaf**, with a separate **key-path internal key** that signs cooperatively when all cosigners are online. The result is a Schnorr signature that looks identical to a single-sig taproot spend on-chain — visually indistinguishable from a wallet of one — while preserving the K-of-N script-path fallback for non-cooperative spends.

This chapter covers the two key-path designs the toolkit supports:

- **NUMS** internal key — a verifiable nothing-up-my-sleeve point that cannot sign; forces all spends through the script path.
- **Designated cosigner** internal key — one cosigner's xpub is the key-path; that cosigner can spend solo via the key path; the others fall back to the script path.

Background — taproot multisig vs. Bitcoin multisig. A pre-taproot 2-of-3 wallet (`wsh(sortedmulti(2,…))`) reveals the cosigner xpubs on every spend, every time. Taproot multisig hides them: the key-path spend reveals only one Schnorr signature; the script-path spend reveals only the cooperating subset's xpubs (the *failure mode* leaks more than the *happy path*). The privacy gain is real but contingent on cooperative spends being the norm.

Examples below use the canonical BIP-39 test vector and two derived test vectors. All three are **public**. Never engrave or fund a wallet built from these seeds.

12.1 Key-path design choice

Internal key	Solo-spendable by	On-chain footprint	When to choose
`nums`	nobody (BIP-341 reference NUMS point)	every spend reveals script	privacy-by-uniformity is less important than guaranteeing K-of-N enforcement on every spend
`@N` (cosigner xpub)	cosigner N alone (key path)	cooperative spend is single-sig-shaped; non-cooperative spend reveals script	privacy in cooperative spends matters; one cosigner is "primary" and the multisig is a recovery / coercion-resistance layer

The two key-path variants are selected differently in `bundle`:

- **NUMS variant** — the *default* for `tr-multi-a` and `tr-sortedmulti-a` templates. Just pass the template; no extra flag is needed. The bundle embeds the BIP-341 reference NUMS point as the internal key.
- **Designated-cosigner variant** — supply a user-defined BIP-388 descriptor via `--descriptor 'tr(@N,sortedmulti_a(K,@0,…))'`, promoting cosigner `@N` out of the script-path leaf set into the key-path. The bundle binds slot `@N` to the chosen cosigner's xpub.

(The `mnemonic export-wallet` subcommand has a `--taproot-internal-key <nums|@N>` flag that toggles the variant on the watch-only export side; this flag does not exist on `bundle` or `verify-bundle`. See the wallet-export workflow.)

12.2 Step 1 — synthesise (NUMS internal key)

```
mnemonic bundle \
  --network mainnet \
  --template tr-sortedmulti-a \
  --threshold 2 \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --slot @1.phrase="legal winner thank year wave sausage worth useful legal winner
   ↪  thank yellow" \
  --slot @2.phrase="letter advice cage absurd amount doctor acoustic avoid letter
   ↪  advice cage above" \
  --self-check
```

The descriptor inside md1 is `tr(NUMS_POINT,sortedmulti_a(2,@0,@1,@2))`. Spends always traverse the script path; the NUMS point cannot sign.

12.3 Step 2 — synthesise (designated cosigner)

```
mnemonic bundle \
  --network mainnet \
  --descriptor 'tr(@2,sortedmulti_a(2,@0,@1))' \
  --slot @0.phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --slot @1.phrase="legal winner thank year wave sausage worth useful legal winner
   ↪  thank yellow" \
  --slot @2.phrase="letter advice cage absurd amount doctor acoustic avoid letter
   ↪  advice cage above" \
  --self-check
```

Cosigner 2's xpub becomes the key-path. The script-path leaf is `sortedmulti_a(2, @0, @1)` — only cosigners 0 and 1 in the leaf, because cosigner 2 has been promoted out. With K=2 and two remaining cosigners, the script path requires unanimity of the script-path participants.

For larger sets, write the descriptor to preserve the original K: e.g. `tr(@4,sortedmulti_a(3,@0,@1` gives a script-path of 3-of-4, still K=3, just over the remaining four cosigners.

12.4 Step 3 — verify and stamp

`mnemonic verify-bundle` accepts the same `--descriptor` (or `--template`) the bundle was built with and re-derives the expected key-path point and script-path leaf to verify the engraved cards match. The stamping ceremony is identical to the non-taproot multisig flow.

12.5 Multi-leaf taproot

For more sophisticated taproot policies — multiple script-path branches, e.g. "2-of-3 of cosigners A/B/C or timelock-recovery key D" — supply a user-defined BIP-388 wallet policy directly:

```
mnemonic bundle \
  --network mainnet \
  --descriptor 'tr(@4,{multi_a(2,@0,@1,@2),and_v(v:older(52596),pk(@3))})' \
  --slot @0.phrase=<…> \
  --slot @1.phrase=<…> \
  --slot @2.phrase=<…> \
  --slot @3.phrase=<…> \
  --slot @4.phrase=<…>
```

Two script-path leaves (cooperative-multisig + emergency-timelock) and a key-path internal key. The toolkit emits the wallet policy bound to the five xpubs in the appropriate slots.

For a deep dive on writing safe multi-leaf taproot policies, the authoritative reference is BIP-388 itself; the descriptors and BIP-388 primer gives a 1-page orientation.

12.6 Variants

- **`tr-multi-a` (unsorted)** — same as `tr-sortedmulti-a` but the cosigner xpubs in the script-path leaf are *not* re-sorted. Use this only if you have a reason to fix cosigner order on-chain.
- **Single-sig taproot** — for a one-cosigner taproot wallet, prefer `--template bip86` (covered in single-sig steel-engraved backup).
- **Tapscript opcodes other than `multi_a`** — supply a `--descriptor` directly. The toolkit's user-supplied descriptor parser accepts the BIP-388 grammar.

Chapter 13

Watch-only xpub bundle

Sometimes you want to *track* a wallet — see its balance, derive addresses, monitor incoming transactions — without holding the seed. The m-format constellation supports this via the **2-card watch-only bundle**: mk1 + md1, no ms1. Without the secret card the wallet cannot sign, but a watch-only client (Bitcoin Core / Sparrow / Specter) can derive addresses and watch the chain.

> **Background — why watch-only matters.** A watch-only wallet sees the blockchain as the spending wallet does, but cannot move funds. Useful for monitoring (incoming-payment tracking, balance reporting), auditing (publishing balance proofs), or coordinator workflows where one party tracks the wallet's UTXOs and presents PSBTs to signers.

13.1 When to build a watch-only bundle

13.2 Building a 2-card watch-only bundle from a phrase

(Same canonical test seed as Chapter 22; see the DANGER box there.)

The two-step shape — first derive the xpub on the seed-holding machine, then build the bundle on a watch-only host — keeps the seed off the bundle-building host:

```
# Step 1 (on the air-gapped seed-holder)
mnemonic convert \
  --from phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --to xpub \
  --template bip84

# Step 2 (on the internet-connected host)
mnemonic bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.xpub=<xpub from Step 1>
```

The output is an mk1 + md1 pair (no ms1, since no seed was passed).

For multisig watch-only, repeat `--slot @N.xpub=…` once per cosigner and pass `--threshold K`:

```
mnemonic bundle \
  --network mainnet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.xpub=<xpub-cosigner-0> \
  --slot @1.xpub=<xpub-cosigner-1> \
  --slot @2.xpub=<xpub-cosigner-2>
```

The output is `3 mk1 + 1 md1 = 4 cards` for a 2-of-3 multisig. Each cosigner can hold one mk1 (their public-key card) plus the shared md1; alternatively, the watch-only operator holds all four.

13.3 Importing a watch-only bundle

For a quick xpub re-derivation from mk1 + md1 alone:

```
# Recover xpub + fingerprint + path from mk1
mnemonic convert \
  --from mk1="<mk1-line-1> <mk1-line-2>" \
  --to xpub --to fingerprint --to path

# Recover wallet-policy template from md1
md decode <md1-line-1> <md1-line-2> <md1-line-3>
```

Compose the result into the watch-only artifact your wallet expects; the wallet-export workflow covers the four supported output formats.

13.4 Privacy-preserving variant

For a watch-only bundle that does not reveal the master fingerprint (useful for coordinator-side bundles that fan out to many cosigners), add `--privacy-preserving`:

```
mnemonic bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.xpub=<xpub> \
  --privacy-preserving
```

The mk1 emits a zero-fingerprint placeholder. Recovery software re-derives the actual fingerprint from the seed at import time, so the privacy property holds only as long as the watch-only bundle is used standalone.

Chapter 14

Recovery paths by damaged-card scenario

Cards get scratched, water-damaged, partly-stamped, lost, or confiscated. This chapter walks through what each failure mode means for the wallet and how to recover, ordered roughly by frequency.

The single principle behind every scenario: **the seed reconstructs everything else**. Public-key material (mk1, md1) is derivable from the seed; only the seed itself is irreplaceable. So recovery paths fan out from "do you still have a seed?".

14.1 Single-sig wallet — single card lost

Lost ms1: re-derive from the seed phrase you remember:

```
mnemonic convert \
  --from phrase="<your phrase>" \
  --to ms1
```

Re-stamp.

Lost mk1: re-derive from the seed:

```
mnemonic convert \
  --from phrase="<your phrase>" \
  --to mk1 \
  --template bip84
```

Re-stamp.

Lost md1: derive a new one from xpub:

```
xpub=$(mnemonic convert --from phrase="<your phrase>" --to xpub --template bip84)
mnemonic bundle \
  --network mainnet \
  --template bip84 \
  --slot @0.xpub=$xpub
```

The `mk1` and `md1` emitted should match the originals (cross-check with `verify-bundle` before discarding the originals); re-stamp the md1 and the mk1 stays intact.

14.2 Single-sig wallet — seed is the only thing left

If you have *only* the seed phrase (no cards), import the phrase into a wallet that supports BIP-39 directly. Standard wallet apps (Bitcoin Core, Sparrow, Electrum, hardware wallets) recover from phrase; the m-format constellation bundle is a *backup*, not a *requirement*.

If you previously used a non-default template (BIP-86 taproot, for example), specify it on import; the seed alone does not record the template, only the m-format md1 does.

14.3 Multisig wallet — one cosigner's ms1 lost

The other cosigners still have their seeds; the shared mk1 set and md1 are intact. Recovery:

1. The cosigner with the missing ms1 re-derives it from their seed (if memorised) and re-stamps.
2. If the cosigner cannot recall their seed but the wallet's *threshold* remains met by the surviving cosigners, the wallet stays spendable. The compromised cosigner is replaced by a fresh key in a follow-up "key rotation" — synthesise a fresh seed, generate a new bundle replacing the lost cosigner's slot, and transfer all funds to the new wallet.

14.4 Multisig wallet — md1 is lost or unreadable

Re-derive from any cosigner's xpub set:

```
mnemonic export-wallet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.xpub=<xpub-0> \
  --slot @1.xpub=<xpub-1> \
  --slot @2.xpub=<xpub-2> \
  --format bip388
```

Or rebuild the full bundle:

```
mnemonic bundle \
  --network mainnet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.xpub=<xpub-0> \
  --slot @1.xpub=<xpub-1> \
  --slot @2.xpub=<xpub-2>
```

The cross-binding `policy_id_stub` re-derives identically as long as the cosigner xpubs and template are unchanged.

14.5 Multisig wallet — partial card damage

The BCH error-correction codec located damage to a small number of characters. The codec's error position diagnostic identifies *which* character is wrong:

```
mnemonic convert --from ms1=ms10entrsqqQqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f --to
↪  phrase
```

Outputs (illustrative):

```
error: ms1 BCH checksum failed
  position 11: invalid character 'Q' (expected 'q')
```

Manually correct the character (the codec narrows the candidate set to typically 1-2 characters at the named position). For damage beyond the codec's correction radius (more than a few errors), the card is unrecoverable from itself — fall back to re-deriving from the seed (or other cosigners' cards in multisig).

14.6 Worst-case scenarios

Scenario	Recoverable?
Single-sig, all cards intact	yes (trivially)
Single-sig, ms1 only	yes — import phrase into BIP-39 wallet
Single-sig, mk1 + md1 only	yes (watch-only); spending requires the seed
Single-sig, no ms1 + no seed	no
2-of-3 multisig, 2 cosigners' ms1s + md1	yes
2-of-3 multisig, 1 cosigner's ms1 + md1	watch-only only; spending requires another seed
2-of-3 multisig, all 3 cosigners lose ms1	no — wallet bricked

14.7 After recovery

Treat the recovered wallet as a *signal* that the original engraving discipline was compromised. After moving funds:

1. Generate a fresh wallet with new entropy.
2. Stamp a new bundle.
3. Move funds.
4. Destroy the damaged plates carefully (they may still contain secrets even though the wallet is empty).

Chapter 15

Migrating from BIP-39-only to the m-format constellation

If you already keep your wallet on a BIP-39 phrase — engraved on steel, stamped on washers, or memorised in your head — the m-format star is a layer *on top of* what you have, not a replacement. The seed phrase remains the secret. The bundle adds checksum-protected encoding, descriptor binding, and a watch-only path that BIP-39 alone does not provide.

This chapter walks the *migration*: keeping your existing seed phrase intact while adding the three-card bundle alongside.

> **Background — what BIP-39 alone misses.** A 12-word phrase recovers a wallet *if and only if* you also remember the derivation. For a default BIP-84 mainnet wallet, most software guesses correctly. For BIP-86 taproot, BIP-49 nested-segwit, a non-zero account index, or any multisig setup, the derivation is unstated by the phrase alone. Worse, transcription errors in steel engraving silently corrupt valid words into different valid words — a 12-word phrase has no per-word checksum. The m-format ms1 card fixes both issues: BCH error correction, plus the policy-side binding that names the wallet's spending rule.

15.1 Migration paths

15.2 Step 1 — identify your wallet's template

Look in your wallet's settings or descriptor export. Common cases:

Wallet UI says	Template flag
"Native SegWit (bech32)" / `bc1q…` addresses	`--template bip84`
"Taproot (bech32m)" / `bc1p…` addresses	`--template bip86`
"Nested SegWit" / `3…` addresses	`--template bip49`
"Legacy" / `1…` addresses	`--template bip44`
Multisig (any)	`--template wsh-sortedmulti` (most common)

Wallet UI says	Template flag

If your wallet exports a BIP-388 wallet policy or a descriptor string directly, prefer that:

```
mnemonic bundle \
  --network mainnet \
  --descriptor 'wpkh([73c5da0a/84h/0h/0h]xpub.../<0;1>/*)' \
  --slot @0.phrase="<your phrase>"
```

The toolkit accepts any BIP-388-conformant descriptor.

15.3 Step 2 — build the bundle

Once the template is known:

```
mnemonic bundle \
  --network mainnet \
  --template <template> \
  --account <account-index> \
  --slot @0.phrase="<your phrase>" \
  --self-check
```

The default `--account 0` matches most wallets. Hardware wallets sometimes use `--account 1` and up; check your device's documentation.

15.4 Step 3 — verify the xpub agrees

The most important migration check: does the toolkit's mk1 carry the *same* xpub your wallet currently uses? Mismatch means the template / account / network is wrong, and a wallet built from the bundle would diverge from your existing wallet — fund-loss risk.

```
# Extract the xpub the bundle would emit
mnemonic convert \
  --from phrase="<your phrase>" \
  --to xpub \
  --template <template> \
  --account <account-index>

# Compare against your wallet's "Show xpub" dialog
```

If they agree, you've matched the wallet. If they don't, iterate on template/account until they do — *do not stamp a bundle whose xpub doesn't match the wallet you intended to back up*.

15.5 Step 4 — stamp the new bundle alongside

The migration is non-destructive: leave the existing BIP-39 plate alone, stamp the three new m-format plates as additional backup. After verification, you can choose

to retire the BIP-39 plate, but there's no rush — the bundle and the BIP-39 plate are interoperable.

15.6 Migrating multisig wallets

If your existing wallet is multisig with cosigners on hardware devices:

1. Each cosigner exports their xpub from their hardware wallet.
2. The coordinator builds the bundle from xpubs only (no phrases):

   ```
   mnemonic bundle \
     --network mainnet \
     --template wsh-sortedmulti \
     --threshold K \
     --slot @0.xpub=<xpub from cosigner 0> \
     --slot @1.xpub=<xpub from cosigner 1> \
     --slot @2.xpub=<xpub from cosigner 2>
   ```

3. Each cosigner additionally produces their own ms1 from their own seed on their own air-gapped device:

   ```
   mnemonic convert --from phrase="<their phrase>" --to ms1
   ```

4. Verify the resulting bundle's xpub set matches the existing wallet's xpub set.

15.7 Things to *not* do during migration

- **Don't share phrases between cosigners during migration.** If the original wallet was set up with each cosigner on their own device, preserve that property. The bundle workflow allows full air-gapped migration; use it.
- **Don't migrate without verifying xpubs.** A wrong-template bundle that ships will look like a backup but recover a different wallet.
- **Don't destroy the BIP-39 plate before round-trip-testing the bundle.** Run `mnemonic verify-bundle` against the engraved cards *and* derive an address from the resulting xpub *and* compare against an existing wallet address before discarding any backup.

Chapter 16

Exporting to Bitcoin Core / BIP-388 / Sparrow / Specter

The bundle's md1 card carries the wallet policy (template + bound xpubs). To turn that into a watch-only wallet your monitoring software can import, run `mnemonic export-wallet`. The subcommand emits the same wallet in any of four interchange formats; pick the one your software wants.

16.1 Format selection

16.2 Bitcoin Core (default format)

Bitcoin Core's `importdescriptors` RPC accepts a JSON array; the toolkit's default output matches:

```
mnemonic export-wallet \
  --template bip84 \
  --slot @0.xpub=<xpub> \
  --network mainnet \
  --range 0,999 \
  --timestamp now
```

Output is a JSON array with the descriptor, its `range`, the `timestamp`, and the `internal` flag for receive vs. change. Pipe into `bitcoin-cli`:

```
bitcoin-cli importdescriptors "$(mnemonic export-wallet --template bip84 --slot
 ↪  @0.xpub=<xpub>)"
```

For a multisig wallet, repeat `--slot @N.xpub` and add `--threshold K`.

The `--bitcoin-core-version` flag controls compatibility: `--bitcoin-core-version 24` emits the older format; `25` (the default) is current.

16.3 BIP-388 wallet policy

Bitcoin Core 24+, Sparrow, Specter, Coldcard, and most modern wallets accept BIP-388 wallet policies:

```
mnemonic export-wallet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.xpub=<xpub-0> \
  --slot @1.xpub=<xpub-1> \
  --slot @2.xpub=<xpub-2> \
  --format bip388
```

Output is the canonical BIP-388 JSON shape:

```
{
  "name": "wsh-sortedmulti-2-of-3",
  "description": "",
  "description_template": "wsh(sortedmulti(2,@0/**,@1/**,@2/**))",
  "keys_info": [
    "[fp0/87h/0h/0h]xpub...",
    "[fp1/87h/0h/0h]xpub...",
    "[fp2/87h/0h/0h]xpub..."
  ]
}
```

(The default `--multisig-path-family bip87` produces `m/87'/0'/0'` paths. For Coldcard / SeedSigner / older-Sparrow compatibility, add `--multisig-path-family bip48` and the paths become `m/48'/0'/0'/2'`.)

16.4 Sparrow + Specter (currently via BIP-388)

`--format sparrow` and `--format specter` are accepted by the binary but currently return a deferral stub:

```
error: --format <sparrow> is deferred to a future release; use
--format bitcoin-core or --format bip388 instead.
```

For now, export as BIP-388 and import via the receiving wallet's BIP-388-aware path:

```
mnemonic export-wallet \
  --template wsh-sortedmulti \
  --threshold 2 \
  --slot @0.xpub=<xpub-0> \
  --slot @1.xpub=<xpub-1> \
  --slot @2.xpub=<xpub-2> \
  --format bip388 \
  --output wallet-policy.json
```

Sparrow consumes wallet-policy JSON via *File → Import → Wallet Policy*. Specter accepts BIP-388 via the *Add Wallet → Import → Multisig* flow. Native Sparrow / Specter shapes will land if a future toolkit release lights up the format stubs.

16.5 From a user-supplied descriptor

If you have a descriptor string that doesn't match a built-in template:

```
mnemonic export-wallet \
  --descriptor 'tr(NUMS,sortedmulti_a(2,@0,@1,@2))' \
  --slot @0.xpub=<xpub-0> \
  --slot @1.xpub=<xpub-1> \
  --slot @2.xpub=<xpub-2> \
  --format bip388
```

The toolkit accepts any BIP-388-conformant descriptor and binds the slotted xpubs into it.

16.6 Taproot multisig export

Taproot multisig requires the `--taproot-internal-key` flag (mirrors `bundle`):

```
mnemonic export-wallet \
  --template tr-sortedmulti-a \
  --threshold 2 \
  --taproot-internal-key nums \
  --slot @0.xpub=<xpub-0> \
  --slot @1.xpub=<xpub-1> \
  --slot @2.xpub=<xpub-2> \
  --format bip388
```

For the cosigner-as-internal-key variant, use `--taproot-internal-key @N`. See Taproot multisig for the design choice.

16.7 Tips

- **Range.** The `--range 0,999` default covers the first 1000 addresses. Increase if you've used more (e.g., a heavily-used exchange wallet). Bitcoin Core re-scans the chain for the imported range; large ranges cost time, not safety.
- **Timestamp.** `--timestamp now` skips re-scan (assumes the wallet has no historical transactions before "now"). Use a unix-seconds value to re-scan from a specific epoch — e.g. `--timestamp 1700000000` for late 2023.
- **Output redirect.** Use `--output file.json` (or `> file.json`) to keep the JSON out of your shell history and ready for piped import.

Chapter 17

Deterministic child secrets via BIP-85

A single seed phrase can mathematically derive an unlimited supply of *new* deterministic secrets — fresh BIP-39 phrases, raw entropy, HD-seed bytes, child xprvs, password strings, even DICE entropy streams. This is BIP-85: take one master seed and grind it through HMAC-SHA-512 with an application code and an index to produce a deterministic, *independent* child secret.

The use cases:

- **Child wallets** — derive an entire new BIP-39 phrase (12 / 18 / 24 words; 9 BIP-85 languages) for an air-gapped hardware wallet without storing a separate backup.
- **Passwords** — derive deterministic per-service passwords; lose the password manager but keep the master seed and you can re-derive every login.
- **Deterministic randomness** — derive entropy for tools that take hex or base64 input.
- **DICE** — derive deterministic *fair-dice* outcomes; useful for collaborative games or audit-replay.

Background — why BIP-85 is safe for the master seed. BIP-85 applies HMAC-SHA-512 to the master node + a hardened path encoding (application + length + index). The output bits are cryptographically independent from the parent seed *and* from each other; observing or even compromising a child secret reveals nothing about the master or about siblings. The master seed is unchanged and unstored alongside the child, so child compromise is contained.

17.1 Subcommand surface

```
mnemonic derive-child \
  --from <FROM> \
  --application <APPLICATION> \
  --length <LENGTH> \
  --index <INDEX>
```

The four required flags:

- **`--from`** — the master source. `--from xprv=<xprv>` (BIP-85 canonical), `--from phrase="<bip39 phrase>"` (combined with `--passphrase` and `--language`), or `--from xprv=-` / `--from phrase=-` to read from stdin.
- **`--application`** — what kind of child secret to derive. Listed below.
- **`--length`** — application-specific size. Range varies; pass `--length 0` for `hd-seed` and `xprv` (length is irrelevant there).
- **`--index`** — hardened child index in `0..2^31`. Increment to derive sibling secrets.

17.2 Applications

`--application`	Output	`--length`
`bip39`	new BIP-39 phrase (9 BIP-85-coded languages)	`12`, `18`, `24`
`hd-seed`	64-byte HD-seed bytes (raw)	`0` (sentinel)
`xprv`	child master xprv	`0` (sentinel)
`hex`	raw hex entropy	`16..=64`
`password-base64`	base64-shaped password	`20..=86`
`password-base85`	base85-shaped password	`10..=80`
`dice`	deterministic dice rolls	`1..=10000`

The seven in-scope BIP-85 applications map to BIP-85 codes 39', 2', 32', 128169', 707764', 707785', and 89101' (DICE; BIP-85 v1.3.0). RSA and RSA-GPG (both under BIP-85 code 828365') are out-of-scope for v0.8 pending RUSTSEC-2023-0071 patch and will land in v0.9.

17.3 Worked examples

(Same canonical test seed as Chapter 22; see the DANGER box there.)

17.3.1 Derive a child BIP-39 phrase

```
mnemonic derive-child \
  --from phrase="abandon abandon abandon abandon abandon abandon abandon abandon
   ↪  abandon abandon abandon about" \
  --application bip39 \
  --length 12 \
  --index 0
```

Output: a fresh 12-word BIP-39 phrase, deterministically derived from the master. Increment `--index` to derive sibling phrases.

17.3.2 Derive a deterministic password

```
mnemonic derive-child \
```

```
  --from phrase="<your master phrase>" \
  --application password-base64 \
  --length 32 \
  --index 0
```

Output: a 32-character base64 password. Use `--index N` for the N-th deterministic password.

17.3.3 Derive a child xprv

```
mnemonic derive-child \
  --from phrase="<your master phrase>" \
  --application xprv \
  --length 0 \
  --index 0
```

Output: a child `xprv...` (or `tprv...` on testnet). Useful for provisioning an air-gapped hardware wallet from a parent backup.

17.3.4 Derive deterministic dice rolls

```
mnemonic derive-child \
  --from phrase="<your master phrase>" \
  --application dice \
  --length 100 \
  --index 0 \
  --dice-sides 6
```

Output: 100 deterministic d6 rolls. The `--dice-sides` flag is required for `dice` and accepts values in `2..=2^32-1`.

17.4 Cross-network derivation

For testnet-targeted children:

```
mnemonic derive-child \
  --from phrase="<phrase>" \
  --application xprv \
  --length 0 \
  --index 0 \
  --network testnet
```

Default is mainnet (matching BIP-85 §"Test Vectors"). Testnet children emit `tprv...` for `xprv` and `c...` WIF for the `hd-seed` application.

17.5 When to use BIP-85 vs. multisig

- **BIP-85 child wallets** — operationally simpler, single backup. Compromise of *any* child reveals nothing about siblings, but compromise of the *master* compromises every child. Best suited for low-value or rotated-frequently child wallets.
- **Multisig** (covered in chapters 32 and 33) — operationally complex, multiple cosigners, threshold-based. Best suited for high-value vaults where compromise of any single secret is recoverable.

The two compose: a multisig wallet whose cosigners' secrets are themselves BIP-85 children of separate masters is a common "deterministic-recovery + threshold" pattern.

Chapter 18

`mnemonic` reference

The integration-layer CLI for the m-format constellation. Five subcommands: `bundle`, `verify-bundle`, `convert`, `export-wallet`, and `derive-child`. Run any with `--help` for the latest flag set; this chapter mirrors v0.8.0.

18.1 `mnemonic bundle`

Synthesise a 3-card engraving bundle from a phrase, entropy, or xpub. Inputs are slotted via `--slot @N.<subkey>=<value>`, repeating.

18.1.1 Synopsis

```
mnemonic bundle --network <NETWORK> [OPTIONS]
```

18.1.2 Flags

Flag	Purpose
`--network <NETWORK>`	mainnet / testnet / signet / regtest
`--template <TEMPLATE>`	bip44 / bip49 / bip84 / bip86 / wsh-multi / wsh-sortedmulti / sh-wsh-multi / sh-wsh-sortedmulti / tr-multi-a / tr-sortedmulti-a
`--descriptor <DESCRIPTOR>`	user-supplied BIP-388 descriptor; mutually exclusive with `--template` and `--descriptor-file`
`--descriptor-file <DESCRIPTOR_FILE>`	descriptor read from a single-line UTF-8 file; mutually exclusive with `--descriptor`
`--language <LANGUAGE>`	BIP-39 wordlist for the input phrase
`--passphrase <PASSPHRASE>`	BIP-39 mnemonic-extension passphrase
`--account <ACCOUNT>`	BIP-32 account index (default 0)

Flag	Purpose
`--json`	emit JSON output
`--no-engraving-card`	suppress the stderr engraving-card layout
`--multisig-path-family <FAMILY>`	bip48 or bip87 (default bip87)
`--privacy-preserving`	suppress the master fingerprint from mk1 + engraving card
`--self-check`	re-parse and verify the emitted bundle round-trips
`--threshold <THRESHOLD>`	multisig K of N (1 ≤ K ≤ N ≤ 16)
`--slot <SLOT>`	repeating; `@N.<subkey>=<value>` (subkey: `phrase`, `entropy`, `xpub`, `fingerprint`, `path`, `wif`, `xprv`)
`--help`	print help

18.1.3 Worked example

See Your first bundle for a single-sig walkthrough; Multi-source 2-of-3 multisig for multisig.

18.2 `mnemonic verify-bundle`

Re-derive expected card content from a seed (or from a partial set of cards) and report per-card pass/fail plus the overall verdict.

18.2.1 Synopsis

```
mnemonic verify-bundle --network <NETWORK> [OPTIONS] [--ms1 ...] [--mk1 ...] [--md1
↪  ...]
```

18.2.2 Flags

Flag	Purpose
`--network <NETWORK>`	mainnet / testnet / signet / regtest
`--template <TEMPLATE>`	as for `bundle`
`--descriptor <DESCRIPTOR>`	user-supplied BIP-388 descriptor
`--descriptor-file <DESCRIPTOR_FILE>`	descriptor read from file
`--threshold <THRESHOLD>`	multisig threshold
`--multisig-path-family <FAMILY>`	bip48 or bip87
`--privacy-preserving`	match a privacy-preserving mk1 emission
`--language <LANGUAGE>`	BIP-39 wordlist

Flag	Purpose
`--passphrase <PASSPHRASE>`	BIP-39 mnemonic passphrase
`--account <ACCOUNT>`	BIP-32 account index
`--slot <SLOT>`	repeating slot input
`--bundle-json <PATH>`	read the bundle from a JSON file emitted by `bundle --json`
`--ms1 <STRING>`	repeating; one ms1 card
`--mk1 <STRING>`	repeating; one mk1 card
`--md1 <STRING>`	repeating; one md1 card
`--json`	JSON output
`--help`	print help

18.2.3 Worked example

See Verifying a bundle.

18.3 `mnemonic convert`

Single-format conversion across the 13-node typed graph: `phrase`, `entropy`, `xpub`, `xprv`, `wif`, `fingerprint`, `path`, `ms1`, `mk1`, `bip38`, `minikey`, `electrum-phrase`, `address`.

18.3.1 Synopsis

```
mnemonic convert --from <NODE>=<value> --to <NODE> [--to <NODE>]... [OPTIONS]
```

18.3.2 Flags

Flag	Purpose
`--from <FROM>`	source node (`phrase=…`, `entropy=…`, `xpub=…`, `xprv=…`, `wif=…`, `ms1=…`, `mk1=…`, `bip38=…`, `minikey=…`, `electrum-phrase=…`); `=-` reads from stdin
`--to <TO>`	target node; repeating for multi-output
`--network <NETWORK>`	mainnet / testnet / signet / regtest
`--template <TEMPLATE>`	as for `bundle`
`--path <PATH>`	derivation path override
`--account <ACCOUNT>`	account index (default 0)
`--language <LANGUAGE>`	BIP-39 wordlist
`--passphrase <PASSPHRASE>`	BIP-39 passphrase
`--passphrase-stdin`	read `--passphrase` from stdin (raw, NULL-byte preserving); BIP-38 V3 use case

Flag	Purpose
`--bip38-passphrase <BIP38_PASSPHRASE>`	distinct BIP-38 Scrypt passphrase channel (v0.8 BREAKING — separate from `--passphrase`)
`--electrum-version <ELECTRUM_VERSION>`	Electrum seed-version selector for `(Entropy, ElectrumPhrase)`
`--electrum-language <ELECTRUM_LANGUAGE>`	Electrum-specific wordlist (English + 4 non-English)
`--fingerprint <FINGERPRINT>`	master fingerprint (input on certain edges)
`--xpub-prefix <XPUB_PREFIX>`	SLIP-0132 prefix selector for emitted xpubs (e.g. zpub, ypub)
`--script-type <SCRIPT_TYPE>`	`p2wpkh` / `p2sh-p2wpkh` / `p2tr` for `(Xpub, Address)` derivation
`--json`	JSON output
`--help`	print help

18.3.3 Worked example

See Minimal recovery walkthrough and Migrating from BIP-39-only to the m-format constellation.

18.4 `mnemonic export-wallet`

Emit watch-only wallet artifacts for Bitcoin Core, BIP-388, Sparrow, or Specter.

18.4.1 Synopsis

```
mnemonic export-wallet [OPTIONS]
```

18.4.2 Flags

Flag	Purpose
`--template <TEMPLATE>`	as for `bundle`
`--descriptor <DESCRIPTOR>`	user-supplied BIP-388 descriptor
`--threshold <THRESHOLD>`	multisig threshold
`--multisig-path-family <FAMILY>`	bip48 or bip87
`--network <NETWORK>`	default mainnet
`--language <LANGUAGE>`	ignored (watch-only); accepted for slot-parser symmetry
`--account <ACCOUNT>`	account index (default 0)
`--slot <SLOT>`	repeating `@N.<subkey>=<value>`

Flag	Purpose
`--format <FORMAT>`	`bitcoin-core` (default) / `bip388` / `sparrow` / `specter`
`--output <OUTPUT>`	output path (`-` = stdout, default)
`--range <RANGE>`	Bitcoin Core `range` field; comma-separated; default `0,999`
`--timestamp <TIMESTAMP>`	Bitcoin Core `timestamp` field; `now` (default) or unix seconds
`--bitcoin-core-version <BITCOIN_CORE_VERSION>`	24 or 25 (default 25)
`--taproot-internal-key <TAPROOT_INTERNAL_KEY>`	`nums` or `@N` for `tr-multi-a` / `tr-sortedmulti-a`
`--help`	print help

18.4.3 Worked example

See Exporting to Bitcoin Core / BIP-388 / Sparrow / Specter.

18.5 `mnemonic derive-child`

BIP-85 deterministic child entropy. Six in-scope applications: `bip39`, `hd-seed`, `xprv`, `hex`, `password-base64`, `password-base85`, plus `dice` (BIP-85 v1.3.0).

18.5.1 Synopsis

```
mnemonic derive-child --from <FROM> --application <APP> --length <LEN> --index <INDEX>
↪  [OPTIONS]
```

18.5.2 Flags

Flag	Purpose
`--from <FROM>`	`xprv=<value>` or `phrase=<bip39>` (with `--passphrase` + `--language`); `=-` to read from stdin
`--application <APPLICATION>`	`bip39` / `hd-seed` / `xprv` / `hex` / `password-base64` / `password-base85` / `dice`
`--length <LENGTH>`	application-specific size; pass `0` for `hd-seed` and `xprv`
`--index <INDEX>`	hardened child index (`0..2^31`)
`--network <NETWORK>`	for `hd-seed` / `xprv` apps; defaults to mainnet

Flag	Purpose
`--language <LANGUAGE>`	BIP-39 wordlist + BIP-85 language code for `--application bip39`
`--passphrase <PASSPHRASE>`	BIP-39 passphrase, only for `--from phrase=…`
`--dice-sides <DICE_SIDES>`	required for `--application dice`; range `2..=2^32-1`
`--help`	print help

18.5.3 Worked example

See Deterministic child secrets via BIP-85.

Chapter 19

`md` (md-cli) reference

The standalone CLI for the md1 format (descriptor-mnemonic / md-codec). Eight subcommands. Most users will use `mnemonic bundle` and `mnemonic verify-bundle` instead; `md` is for direct descriptor-card inspection or when integrating md1 into a non-toolkit pipeline.

Every subcommand below accepts `--help` (`-h`) for inline help.

19.1 `md encode`

Encode a BIP-388 wallet-policy template into one or more md1 backup strings.

19.1.1 Synopsis

```
md encode [OPTIONS] [TEMPLATE]
```

19.1.2 Flags

Flag	Purpose
`--from-policy <EXPR>`	compile a sub-Miniscript-Policy expression into a template
`--context <CTX>`	script context (`tap` / `segwitv0`) for `--from-policy`
`--path <PATH>`	override the inferred shared derivation path
`--key <@i=XPUB>`	concrete xpub for placeholder `@i`; repeatable
`--fingerprint <@i=HEX>`	master-key fingerprint for placeholder `@i`; repeatable
`--network <NETWORK>`	xpub validation + JSON output labelling

Flag	Purpose
`--force-chunked`	force chunked encoding even for short policies
`--force-long-code`	force the long BCH code even when the regular suffices
`--policy-id-fingerprint`	print the freshly-computed PolicyId fingerprint after the phrase
`--json`	emit JSON output

19.1.3 Worked example

```
md encode 'wpkh(@0/<0;1>/*)'
# md1qqpqqxqxkceprx7rap4t
```

19.2 `md decode`

Decode one or more md1 backup strings into a wallet-policy template.

19.2.1 Synopsis

```
md decode [OPTIONS] <STRINGS>...
```

19.2.2 Flags

Flag	Purpose
`--json`	emit JSON output

19.2.3 Worked example

See Minimal recovery walkthrough Step 3.

19.3 `md inspect`

Decode + pretty-print everything the codec sees (for debugging).

19.3.1 Synopsis

```
md inspect [OPTIONS] <STRINGS>...
```

19.3.2 Flags

Flag	Purpose
`--json`	emit JSON output

19.4 `md address`

Derive bitcoin addresses from a wallet-policy-mode descriptor (md1 phrases or template + keys).

19.4.1 Synopsis

```
md address [OPTIONS] <PHRASES|--template <TEMPLATE>>
```

19.4.2 Flags

Flag	Purpose
`--template <TEMPLATE>`	BIP-388 template; requires `--key`; mutually exclusive with positional phrases
`--key <@i=XPUB>`	concrete xpub for placeholder `@i`; requires `--template`; repeatable
`--fingerprint <@i=HEX>`	master-key fingerprint for placeholder `@i`; requires `--template`
`--network <NETWORK>`	xpub validation and address rendering
`--chain <CHAIN>`	multipath alternative selector (0 = receive, 1 = change)
`--change`	sugar for `--chain 1`
`--index <INDEX>`	starting index along the wildcard
`--count <COUNT>`	number of consecutive addresses
`--json`	emit JSON output

19.4.3 Worked example

```
md address md1zsxdspq...  # decode + derive address from existing card
```

19.5 `md bytecode`

Dump the raw payload bits in an annotated layout (debugging / auditing).

19.5.1 Synopsis

```
md bytecode [OPTIONS] <STRINGS>...
```

19.5.2 Flags

Flag	Purpose
`--json`	emit JSON output

19.6 `md compile`

Compile a sub-Miniscript-Policy expression into a BIP-388 template.

19.6.1 Synopsis

```
md compile [OPTIONS] <EXPR>
```

19.6.2 Flags

Flag	Purpose
`--context <CTX>`	script context (`tap` / `segwitv0`)
`--json`	emit JSON output

19.7 `md vectors`

Regenerate the project's test-vector corpus (maintainer tool; end-users rarely run this).

19.7.1 Synopsis

```
md vectors [--out <DIR>]
```

19.7.2 Flags

Flag	Purpose
`--out <DIR>`	output directory for regenerated vectors

19.8 md verify

Verify md1 strings re-encode to a given template.

19.8.1 Synopsis

```
md verify --template <TEMPLATE> [OPTIONS] <STRINGS>...
```

19.8.2 Flags

Flag	Purpose
`--template <TEMPLATE>`	BIP-388 template to verify against
`--key <@i=XPUB>`	concrete xpub for placeholder `@i`; repeatable
`--fingerprint <@i=HEX>`	master-key fingerprint for placeholder `@i`
`--network <NETWORK>`	xpub validation network

Chapter 20

`ms` (ms-cli) reference

The standalone CLI for the ms1 format (mnemonic-secret / ms-codec). Five subcommands. Most users will use `mnemonic bundle` instead; `ms` is for direct secret-card inspection or for ms1-only pipelines.

ms1 adopts BIP-93 codex32 directly via the upstream `rust-codex32` crate (unlike md1 / mk1, which fork the BCH plumbing); `ms` is a thin wrapper exposing the codex32 single-string mode in a toolkit-style CLI.

Every subcommand below accepts `--help` (`-h`) for inline help.

20.1 `ms encode`

Encode a BIP-39 mnemonic (or hex entropy) as an ms1 string.

20.1.1 Synopsis

```
ms encode [OPTIONS] <--phrase <PHRASE> | --hex <HEX>>
```

20.1.2 Flags

Flag	Purpose
`--phrase <PHRASE>`	BIP-39 mnemonic; `-` reads from stdin
`--hex <HEX>`	hex-encoded entropy bytes (16/20/24/28/32 bytes = 32/40/48/56/64 hex chars)
`--language <LANGUAGE>`	BIP-39 wordlist for the input phrase; ignored under `--hex`
`--no-engraving-card`	suppress the stderr engraving-card layout
`--json`	emit a single JSON object on stdout

20.1.3 Worked example

```
ms encode --phrase "abandon abandon abandon abandon abandon abandon abandon abandon
 ↪  abandon abandon abandon about"
# ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f
```

20.2 `ms decode`

Decode an ms1 string back to its BIP-39 mnemonic and entropy bytes.

20.2.1 Synopsis

```
ms decode [OPTIONS] <STRING>
```

20.2.2 Flags

Flag	Purpose
`--language <LANGUAGE>`	BIP-39 wordlist for the output phrase
`--json`	emit JSON output

20.2.3 Worked example

```
ms decode ms10entrsqqqqqqqqqqqqqqqqqqqqqqqqqqqqcj9sxraq34v7f
# phrase: abandon abandon abandon … about
# entropy (hex): 00000000000000000000000000000000
```

20.3 `ms inspect`

Inspect an ms1 string's structural fields and decoder verdict (debug-style verbose output).

20.3.1 Synopsis

```
ms inspect [OPTIONS] <STRING>
```

20.3.2 Flags

Flag	Purpose
`--json`	emit JSON output

20.4 ms verify

Verify an ms1 string is valid (and optionally round-trips against a phrase).

20.4.1 Synopsis

```
ms verify [OPTIONS] <STRING>
```

20.4.2 Flags

Flag	Purpose
`--phrase <PHRASE>`	optional BIP-39 phrase to round-trip against
`--language <LANGUAGE>`	BIP-39 wordlist for `--phrase`
`--json`	emit JSON output

20.5 ms vectors

Print the SHA-pinned v0.1 test-vector corpus as JSON (for downstream test-suite consumption).

20.5.1 Synopsis

```
ms vectors [--pretty]
```

20.5.2 Flags

Flag	Purpose
`--pretty`	pretty-print the JSON output

Chapter 21

`mk` (mk-cli) reference

The standalone CLI for the mk1 format (mnemonic-key / mk-codec). Five subcommands. Most users will use `mnemonic bundle` and `mnemonic verify-bundle` instead; `mk` is for direct key-card inspection, mk1-plate recovery from an air-gapped machine without shipping the secret-material code paths of the toolkit, or when integrating mk1 into a non-toolkit pipeline.

`mk-cli` ships in the `bg002h/mnemonic-key` repo as a separate binary alongside the `mk-codec` library; install with `cargo install --git https://github.com/bg002h/mnemonic-key --tag mk-cli-v0.2.0 --bin mk`.

Every subcommand below accepts `--help` (`-h`) for inline help.

21.1 `mk encode`

Encode an xpub plus origin metadata (master fingerprint + derivation path) and one or more `policy_id_stub` values into one or more mk1 backup strings.

21.1.1 Synopsis

```
mk encode --xpub <XPUB> --origin-path <PATH> [OPTIONS]
```

21.1.2 Flags

Flag	Purpose
`--xpub <XPUB>`	BIP-32 extended public key (xpub-prefixed string)
`--origin-fingerprint <HEX>`	8-hex-char master fingerprint; mutually exclusive with `--privacy-preserving`
`--origin-path <PATH>`	derivation path (e.g., `m/84'/0'/0'`)
`--policy-id-stub <HEX>`	8 hex chars (4 bytes) for one stub; repeatable

Flag	Purpose
`--from-md1 <MD1-STRING>`	derive a stub from an md1 wallet-policy string; repeatable
`--privacy-preserving`	emit without master fingerprint; mutually exclusive with `--origin-fingerprint`
`--force-chunked`	force chunked output (reserved; codec auto-dispatches)
`--force-long-code`	force long-code BCH variant (reserved; codec auto-dispatches)
`--json`	emit a single JSON object on stdout

At least one of `--policy-id-stub` or `--from-md1` is required (the underlying `KeyCard.policy_id_stubs` slot must be non-empty).

21.1.3 Worked example

```
mk encode \
  --xpub
   ↪  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXY
   ↪  \
  --origin-fingerprint 73c5da0a \
  --origin-path "m/84'/0'/0'" \
  --from-md1 md1zsxdspqqqpm6jzzqqvqz6qu79mg9p2sgfff6p2eph8wftp5uf6gqnlgzqqqnymv0
```

21.1.4 Output

Text mode emits one mk1 string per line, one per chunk. JSON mode emits

```
{
  "schema_version": 1,
  "mk1_strings": ["mk1...", "mk1..."],
  "chunk_count": 2,
  "code_variant": "regular"
}
```

`code_variant` is `regular` or `long` per the BCH-code dispatch.

21.2 `mk decode`

Reassemble + decode one or more mk1 strings into the underlying xpub + origin + policy-id-stub fields.

21.2.1 Synopsis

```
mk decode [OPTIONS] <MK1-STRING>...
```

Use `-` as a positional argument to read one mk1 string per line from stdin.

21.2.2 Flags

Flag	Purpose
`--json`	emit JSON output

21.2.3 Worked example

```
mk decode \
  ↪  mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8q
  ↪  \
   mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh
```

21.2.4 Output

Text mode:

```
xpub:
  ↪  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXYVU
origin_fingerprint:  73c5da0a
origin_path:         m/84'/0'/0'
policy_id_stubs:     deadbeef
chunks:              2 (regular)
```

`origin_fingerprint` reads `(omitted, privacy-preserving mode)` for cards encoded under `--privacy-preserving`.

JSON mode:

```
{
  "schema_version": 1,
  "xpub": "xpub6...",
  "origin_fingerprint": "73c5da0a",
  "origin_path": "m/84'/0'/0'",
  "policy_id_stubs": ["deadbeef"],
  "chunks": 2,
  "code_variant": "regular"
}
```

`origin_fingerprint` is `null` for privacy-preserving cards.

21.3 `mk inspect`

Decode + structural commentary: the xpub-derived fingerprint, each derivation-path component spelled out (hardened vs. normal), per-chunk BCH-code variant. v0.2 inspect is intentionally less rich than `md inspect` because mk-codec's bytecode-layer surface is not yet public (deferred to a v0.3 cycle alongside the public-bytecode-API decision).

21.3.1 Synopsis

```
mk inspect [OPTIONS] <MK1-STRING>...
```

21.3.2 Flags

Flag	Purpose
`--json`	emit JSON output

21.3.3 Output

Text mode adds the following fields beyond `mk decode` text output:

```
xpub_fingerprint:    73c5da0a
  component[0]:       84h (hardened)
  component[1]:       0h (hardened)
  component[2]:       0h (hardened)
  chunk[0]:           regular (BCH variant)
  chunk[1]:           regular (BCH variant)
```

JSON mode adds `xpub_fingerprint`, `origin_path_components` (array of strings), and `chunk_variants` (array of `"regular"|"long"`).

21.4 `mk verify`

Verify mk1 strings decode cleanly (BCH check + structural validity) and optionally cross-check against expected field values.

21.4.1 Synopsis

```
mk verify [OPTIONS] <MK1-STRING>...
```

21.4.2 Flags

Flag	Purpose
`--xpub <EXPECTED-XPUB>`	assert decoded xpub equals this
`--origin-fingerprint <EXPECTED-HEX>`	assert decoded fingerprint equals this
`--origin-path <EXPECTED-PATH>`	assert decoded path equals this
`--policy-id-stub <HEX>`	assert decoded stubs match this set; repeatable, order-sensitive
`--from-md1 <MD1-STRING>`	derive expected stub from md1 string; repeatable, order-sensitive
`--json`	emit a JSON envelope on stdout

Without any expected-* flags, `mk verify` performs BCH-checksum and structural validation only. With expected flags, it additionally asserts content equality on each supplied field.

21.4.3 Worked example

```
mk verify \
  --xpub
   ↪  xpub6CatWdiZiodmUeTDp8LT5or8nmbKNcuyvz7WyksVFkKB4RHwCD3XyuvPEbvqAQY3rAPshWcMLoP2fMFMKHPJ4ZeZXY
   ↪  \
  --origin-fingerprint 73c5da0a \
  --origin-path "m/84'/0'/0'" \

 ↪  mk1qprsqhpqqsq3cqtsleeutks2qvzg3vs70mejhk622ws2kgdemj2cd8zwj2skzx2wq0qw70l4q99vdyh5x0z8v4yslsp8q
 ↪  \
  mk1qprsqhpp0f30mtxzd65mvwcur9usdatwuqvq6z70r9nwrgk6xn6l8gy6nwa2n977sw6zh34rma0nh
```

Exits 0 with `OK: mk1 string(s) decode cleanly ...` on success, exits 4 with a `ContentMismatch` error envelope when an expected-* flag disagrees with the decoded value.

21.5 `mk vectors`

Print the SHA-pinned mk-codec v0.1 test-vector corpus as JSON.

21.5.1 Synopsis

```
mk vectors [OPTIONS]
```

21.5.2 Flags

Flag	Purpose
`--pretty`	indent the JSON output for human readability (ignored when `--out` is set)
`--out <DIR>`	write one `<name>.json` per fixture into `<DIR>` instead of emitting to stdout

The corpus is `include_str!`-baked into the binary at build time, so `cargo install`-style installs are fully self-contained: `mk vectors` runs from any working directory without a fixture-path dependency.

21.5.3 Worked example

```
mk vectors --out /tmp/mk-vectors
# (stderr) wrote N vector file(s) to /tmp/mk-vectors
```

21.6 Error envelope

In `--json` mode, errors are emitted as a structured envelope:

```
{
  "schema_version": 1,
  "error": {
    "kind": "<error-kind-name>",
    "message": "<human-readable>",
    "exit_code": 2,
    "details": { }
  }
}
```

Error `kind` values map 1:1 to mk-codec's `Error` enum variants (e.g., `InvalidHrp`, `MixedCase`, `BchUncorrectable`, `ChunkSetIdMismatch`, `PathTooDeep`) plus mk-cli-only kinds: `UsageError`, `ContentMismatch`, `IoError`, `MdCodec` (`--from-md1` parse failures), and `FutureFormat` (reserved for v0.3+ when mk-codec distinguishes future-version strings from currently-unsupported ones). The `details` field is kind-specific (e.g., `ContentMismatch` carries `{field, expected, actual}`).

21.7 Exit codes

Code	Meaning
0	Success.
2	mk1 format violation; codec rejected the input. Maps to `Error::*` kinds except `UnsupportedVersion`.
3	FutureFormat — string is well-formed but its declared version is newer than this tool. Maps to `Error::UnsupportedVersion`.
4	Verify content mismatch (only `mk verify` with expected-* flags emits this).
64	CLI usage error per clap convention (unrecognized flag, missing required argument, etc.).

Chapter 22

Format-by-format decision table

When does each format earn its place? This chapter lays out the four m-format constellation surfaces side by side so a reader can pick *which* format a given task targets — and then jump to the right tool.

22.1 The four surfaces

Format	Carries	Reach (network)	Standalone CLI	Library
`ms1`	BIP-39 entropy	mainnet / testnet / signet / regtest	`ms`	`ms-codec` (Rust)
`mk1`	xpub + origin (fingerprint + path)	mainnet / testnet / signet / regtest	(none in v0.1)	`mk-codec` (Rust)
`md1`	wallet policy (template + bound xpub)	mainnet / testnet / signet / regtest	`md`	`md-codec` (Rust)
`mnemonic-toolkit`	integration over the three card formats	all four; multi-source slot inputs	`mnemonic`	`mnemonic-toolkit` (Rust)

22.2 Pick by task

Task	Use
Engrave a single-sig BIP-84 wallet	`mnemonic bundle --template bip84 --slot @0.phrase=…`

Task	Use
Engrave a 2-of-3 multisig wallet	`mnemonic bundle --template wsh-sortedmulti --threshold 2 --slot @0.phrase=… --slot @1.phrase=… --slot @2.phrase=…`
Engrave a taproot multisig	`mnemonic bundle --template tr-sortedmulti-a --taproot-internal-key nums …`
Re-derive a phrase from an ms1 card	`mnemonic convert --from ms1=… --to phrase` (or `ms decode <STRING>`)
Re-derive xpub + path from an mk1 card	`mnemonic convert --from mk1=… --to xpub --to fingerprint --to path`
Decode a wallet policy from md1	`md decode <STRINGS>` (positional)
Cross-check a 3-card bundle	`mnemonic verify-bundle …`
Watch-only export (Bitcoin Core / BIP-388 / Sparrow / Specter)	`mnemonic export-wallet …`
Derive a child secret (BIP-85)	`mnemonic derive-child …`

22.3 When you don’t need the toolkit

Some workflows are simpler with the standalone CLIs:

- **Just an ms1 round-trip.** Use `ms encode` + `ms decode`. No toolkit dependency.
- **Just an md1 round-trip.** Use `md encode` + `md decode` or `md verify`. No toolkit dependency.
- **Library integration in Rust.** Use the codec crates directly; the toolkit is an integration-and-CLI layer atop them.

22.4 When you need the toolkit

- **Multi-card bundles.** The toolkit synthesises and verifies ms1+mk1+md1 together; the standalone CLIs each see only their own format.
- **Cross-binding verification.** `policy_id_stub` cross-binding is computed by the toolkit, not by the standalone CLIs.
- **Multi-source multisig.** Each cosigner’s seed → bundle synthesis is a toolkit feature; the standalone CLIs handle one card at a time.
- **Wallet export.** `mnemonic export-wallet` produces Bitcoin Core / BIP-388 / Sparrow / Specter artifacts; standalone codecs produce card strings only.
- **BIP-85.** `mnemonic derive-child` is toolkit-only.

The next chapters compare narrower pairs: `mnemonic` vs `ms`, `mnemonic` vs `md`, m-format constellation vs other backup standards, single-sig vs multisig, BIP-39 vs BIP-38 passphrases, Bitcoin Core importdescriptors vs BIP-388.

Chapter 23

`mnemonic` vs `ms-cli` for secret cards

For ms1 (the secret card alone), both `mnemonic` and `ms` will do the job. When does each fit?

23.1 Use `ms` (the standalone CLI) when

- You want a *single-card* round-trip (encode a phrase to ms1, or decode an ms1 to a phrase) with no toolkit dependency.
- You're integrating ms1 into a non-toolkit pipeline — a backup script, a hardware-wallet firmware build, a third-party recovery tool.
- You want a debug-friendly inspect view (`ms inspect`) of the raw codex32 fields. The toolkit doesn't expose this.
- You need the canonical SHA-pinned v0.1 test-vector corpus (`ms vectors --pretty`).

23.2 Use `mnemonic` (the toolkit) when

- You need ms1 alongside mk1 and md1 in a coherent bundle.
- You want the cross-binding `policy_id_stub` verification across the three cards.
- You want `mnemonic verify-bundle` to assert the ms1 entropy round-trips against the rest of the bundle.
- You want Bitcoin Core / BIP-388 / Sparrow / Specter wallet-export artifacts (`mnemonic export-wallet`).
- You want BIP-85 child derivations (`mnemonic derive-child`).
- You want the unified `--slot @N.<subkey>=<value>` input shape that composes with multisig.

23.3 Side-by-side

Capability	`ms encode` / `ms decode`	`mnemonic bundle` / `mnemonic convert`
BIP-39 phrase ↔ ms1	yes	yes
Hex entropy ↔ ms1	yes	yes (via `--slot @0.entropy=…`)
10 BIP-39 wordlists	yes	yes
BCH error-position diagnostic	yes (decode + inspect)	yes (verify-bundle)
Cross-binding to mk1 + md1	no	yes
Multisig (multi-source ms1)	no	yes
Watch-only wallet export	no	yes
BIP-85 derivation	no	yes
Stable API for non-toolkit pipelines	yes (smaller dep)	toolkit pulls in many crates

23.4 Practical takeaway

`ms` is the right tool for an ms1-only pipeline (e.g., a paper wallet generator that only emits the secret card). `mnemonic` is the right tool for end-user workflows that touch the bundle as a whole.

Chapter 24

`mnemonic` vs `md-cli` for descriptor cards

For md1 (the descriptor card), both `mnemonic` and `md` cover the core needs but with different specialities.

24.1 Use `md` (the standalone CLI) when

- You want a *direct* descriptor → md1 conversion (`md encode`) or md1 → descriptor conversion (`md decode`).
- You want to compile a Miniscript Policy expression into a BIP-388 template (`md compile`).
- You need to derive concrete addresses from a wallet-policy descriptor (`md address` — receive/change selectors, count + index windowing).
- You want to inspect the raw payload bits of an md1 string for debugging (`md bytecode`).
- You want to regenerate the project's pinned test vectors (`md vectors --out` — maintainer use).
- You're verifying md1 strings against a known template (`md verify`).
- You're integrating md1 into a non-toolkit pipeline.

24.2 Use `mnemonic` (the toolkit) when

- You need md1 alongside ms1 and mk1 in a bundle.
- You want the cross-binding `policy_id_stub` to verify md1 against the seed-and-key side automatically.
- You're building a watch-only artifact for Bitcoin Core / BIP-388 / Sparrow / Specter (`mnemonic export-wallet` consumes md1's role internally).
- You're tracking a multi-source multisig flow where md1 is one of many cards.

24.3 Side-by-side

Capability	`md` standalone	`mnemonic` toolkit
Encode descriptor → md1	yes (`md encode`)	yes (via `bundle`)
Decode md1 → descriptor	yes (`md decode`)	yes (via `bundle` round-trip)
Compile Policy → template	yes (`md compile`)	no (only template names)
Derive addresses	yes (`md address`)	no (use `md address` separately)
Inspect raw bytes	yes (`md bytecode`, `md inspect`)	no
Verify against template	yes (`md verify`)	yes (via `verify-bundle`)
Bundle synthesis with ms1 + mk1	no	yes
Cross-binding `policy_id_stub`	no	yes
Wallet export	no	yes

24.4 Practical takeaway

`md` is the right tool for descriptor-side work — Policy compilation, address derivation, raw-byte inspection. `mnemonic` takes md1 as part of a bundle but doesn't replicate the descriptor-side power tools that `md` provides directly. Many workflows use both: `md compile` to land a template, then `mnemonic bundle --descriptor=…` to ship it across all three cards.

Chapter 25

m-format constellation vs SLIP-39 vs naked BIP-39 vs Shamir

Bitcoin self-custody has a small handful of mature seed-backup standards. None is universally best; each has a use case for which it was designed. This chapter compares the four most common: naked BIP-39, the m-format constellation, SLIP-39 / Shamir-style splits, and codex32 alone (without a card-bundle wrapper).

The intent here is **scope** rather than *ranking*. A reader's right choice depends on threat model, recovery skills, and the wallet's target lifetime.

25.1 Side-by-side

	Naked BIP-39	m-format constellation	SLIP-39 / Shamir	codex32 alone
Per-character checksum	no	yes (BCH)	partial	yes (BCH)
Engraving-friendly alphabet	no (English wordlist)	yes (32-char)	partial	yes (32-char)
K-of-N share splitting	no	v0.2 (planned)	yes	yes
Wallet-policy binding	no	yes (md1)	no	no
Multisig coordination	external	yes (multi-source)	no	no
Library availability	wide (every wallet)	narrow (toolkit)	medium (Trezor, others)	medium (rust-codex32)

	Naked BIP-39	m-format constellation	SLIP-39 / Shamir	codex32 alone
Hardware wallet first-class	yes	no (yet)	yes (Trezor)	no
Ceremony complexity	low	medium-high	medium	low
BIP standardisation	BIP-39	none yet (BIP-draft)	SLIP-39 (Trezor)	BIP-93

25.2 When each fits

Naked BIP-39 — the path of least resistance for new users, with a hardware-wallet ecosystem that already speaks it. The cost is brittleness: no per-character checksum, no policy binding, and steel engravings of 12-24 words have failed in the field due to mis-stamping.

The m-format constellation — the natural fit when (a) the wallet is multisig or non-default-template, (b) the backup must be self-describing (template + xpubs travel with the seed), and (c) the user values BCH error correction over hardware-wallet support. The cost is operational complexity: three plates, a ceremony, and a toolkit dependency.

SLIP-39 / Shamir-style splits — the natural fit when the goal is *threshold secret-sharing* of a single seed across many parties or locations, *and* the wallet is single-sig. SLIP-39 specifies the share-encoding scheme; many hardware wallets implement it. The cost is that SLIP-39 doesn't carry policy or xpub binding, so multisig recovery still needs an out-of-band descriptor.

codex32 alone — the natural fit when the only need is "BIP-39 entropy with BCH error correction, no multisig, no descriptor." This is essentially the m-format constellation's ms1 card without the bundle. Useful for ms1-only pipelines (paper wallets, hardware-wallet firmware) where the toolkit is overkill.

25.3 Composability

The four standards are not mutually exclusive. Useful combinations in production:

- **m-format constellation + hardware wallet.** The hardware wallet handles daily signing from BIP-39; the m-format bundle backs up the seed *and* the policy binding for disaster recovery.
- **SLIP-39 + the m-format mk1+md1 cards.** SLIP-39 splits the secret across N locations; the mk1 + md1 cards provide the watch-only side without a full toolkit dependency.
- **Naked BIP-39 + paper-walleted codex32.** The codex32 ms1 card serves as the verification mirror for a BIP-39 phrase engraved elsewhere; mismatches between the two surface as decode errors.

25.4 Choosing for the long term

For a wallet meant to outlive the operator (estate planning, family trusts), favour standards that are widely re-implemented and unlikely to disappear:

Likelihood of long-term software support	Standard
highest	BIP-39 (universal)
high	SLIP-39 (Trezor + others)
high	BIP-93 codex32 (BIP-tracked)
medium	m-format constellation (one toolkit, BIP-draft underwa

A hybrid approach — e.g., naked BIP-39 phrase plus an m-format bundle as the “self-describing” supplement — minimises the bet on any one tool’s continued maintenance.

Chapter 26

Single-sig vs multisig vs threshold-secret decision tree

Three orthogonal axes for backing up Bitcoin: how many *seeds* are involved (single vs many), how many *signatures* are needed (single vs threshold), and how the *secret material itself* is split (monolithic vs share-split). The m-format constellation supports all three combinations; choosing wrong is operationally expensive.

26.1 Decision tree

26.2 The three axes

Axis	Single	Multi
Number of seeds	one BIP-39 phrase	several phrases, one per cosigner
Number of signatures	one (single-sig)	K of N (multisig threshold)
Secret material structure	monolithic phrase or ms1	share-split (SLIP-39 / codex32 K-of-N)

Most production setups pick on each axis independently. The m-format star covers:

- **single-sig + monolithic** — single ms1 + single mk1 + md1 — the simplest case (chapter 31).
- **multisig + monolithic** — multiple ms1s + multiple mk1s + shared md1 — the headline use case (chapter 32).
- **single-sig + share-split** — planned for ms-codec v0.2 (codex32 K-of-N).
- **multisig + share-split** — composable post-v0.2: each cosigner's ms1 can itself be share-split.

26.3 When single-sig is enough

- The wallet's value is below the operational complexity threshold of multisig (each operator's risk tolerance differs; pick a number).
- Recovery training and ongoing operations are constrained by a single human's discipline.
- The seed will be backed up by physical separation alone, not cryptographic threshold.

A well-engraved single-sig steel backup *with* an m-format bundle (adding BCH error correction and policy binding) often beats a multisig setup whose cosigners aren't disciplined about ceremony.

26.4 When multisig earns its complexity

- The wallet's value justifies recurring operational cost (each cosigner runs through verify-bundle when receiving the bundle, participates in PSBTs, etc.).
- The threat model includes single-point seed compromise (theft, coercion, key-logger).
- The cosigners are independent humans / locations / devices.

A 2-of-3 multisig is the canonical starting point: tolerates one cosigner unavailability without bricking the wallet.

26.5 When threshold-secret-splitting is the right axis

- The threat is *single-point physical-medium loss*, not single-point human compromise.
- The wallet is single-sig (or each cosigner of a multisig wants their own share-split).
- The recipients of the shares are trusted parties / locations but not multisig cosigners.

Until ms-codec v0.2 lands, threshold-secret-splitting in the m-format ecosystem is via SLIP-39 (engineering effort to interoperate) or via standalone codex32 K-of-N (rust-codex32 exposes share modes); the upcoming v0.2 of ms-codec will surface this through `mnemonic` directly.

26.6 Anti-patterns

- **Multisig used as "split single-sig"** — engraving each seed of a multi-seed setup but storing all seeds in one location defeats multisig's purpose. Geographic separation is non-optional.
- **Single-sig with the seed engraved on multiple plates** — the attacker who finds *one* plate compromises everything. Use threshold-secret-splitting (SLIP-39 / codex32 v0.2) or single- cosigner with off-site backups, not duplicated full plates.

- **Multisig at a threshold that requires every cosigner** (K=N) — loses multisig's recovery property. Always pick K < N for any K > 1; if you genuinely need K=N, single-sig with stricter ceremony is operationally simpler.

Chapter 27

BIP-39 passphrase vs BIP-38 passphrase

Two unrelated standards both call their additional secret a "passphrase," and they do *very* different things. Confusing them costs funds. Toolkit v0.8 made the distinction explicit by splitting them into two separate CLI flags.

27.1 The two standards

Standard	What the passphrase protects	KDF	Composes with
BIP-39	The seed-derivation step	PBKDF2-HMAC-SHA-512, 2048 iterations	a 12-/24-word mnemonic
BIP-38	A single-key WIF	Scrypt (memory-hard)	one privkey at a time, not a wallet

A BIP-39 passphrase becomes part of the wallet's identity: with one phrase + one passphrase you have one wallet; with the same phrase + a different passphrase you have a different wallet. There's no way to test a passphrase without trying it.

A BIP-38 passphrase is encryption-at-rest for one private key. The encrypted form (`6P...`) is decryptable only with the right passphrase; the wrong passphrase produces a wrong key (silent failure, then funds going to nowhere on first send).

27.2 The toolkit v0.8 BREAKING change

In toolkit v0.7, `mnemonic convert --passphrase` covered both purposes. On a `(phrase, ms1)` edge it fed BIP-39 PBKDF2; on a `(wif, bip38)` edge it fed BIP-38 Scrypt. On a *composite* edge like `(phrase, bip38)` (encrypt the BIP-39-derived WIF

with BIP-38 Scrypt), the same flag was reused for both KDFs — operationally ambiguous.

v0.8 split the two:

```
mnemonic convert \
  --from phrase="…" \
  --to bip38 \
  --passphrase "<BIP-39 passphrase>" \
  --bip38-passphrase "<BIP-38 passphrase>"
```

The two arguments are now independent. On composite edges, both passphrases must be supplied (or both can be empty). On direct `(wif, bip38)` and `(bip38, wif)` edges, `--bip38-passphrase` falls back to `--passphrase` if the dedicated flag is absent.

Edge	`--passphrase`	`--bip38-passphrase`
`(phrase, ms1)`	BIP-39	unused
`(wif, bip38)` direct	falls through to BIP-38 if `--bip38-passphrase` unset	BIP-38
`(phrase, bip38)` composite	BIP-39	BIP-38 (independent; no fallback)
`(entropy, bip38)` composite	unused (no BIP-39 step)	BIP-38 (defaults to "" if unset; BREAKING)

27.3 NULL-byte passphrase edge case

BIP-38 V3 (the encrypted-private-key spec) explicitly allows passphrases containing `U+0000` (NULL). POSIX argv cannot carry NULL bytes (the C runtime's argv parser terminates each argument at NULL). So passing a NULL-byte passphrase via `--passphrase` or `--bip38-passphrase` is impossible.

v0.8 added `--passphrase-stdin` to close this gap:

```
printf '\x00secret' | mnemonic convert \
  --from wif="…" \
  --to bip38 \
  --passphrase-stdin
```

The flag tells the binary to read `--passphrase`'s value from stdin (raw, no newline-stripping, no UTF-8 normalisation), preserving NULL bytes. Mutually exclusive with `--passphrase`.

This was driven by the v0.7.1 BIP-38 test-vector audit cycle, which exposed a third-party test vector (BIP-38 V3 NULL-byte passphrase) that the toolkit could not reproduce because of the argv NULL limitation. v0.8 fixed it.

27.4 Practical recommendations

- **Use a BIP-39 passphrase only if you understand its always-mandatory nature.** Forgetting the passphrase loses the wallet, and there's no way to check "is this the right one?" short of trying to derive an address you've previously used.
- **Use a BIP-38 passphrase for one-off-key paper wallets, never for HD wallet backup.** BIP-38 was designed for "import this one WIF into your wallet"; HD wallets and the m-format constellation are for multi-key setups.
- **If your software offers "extra word" or "13th word" prompts, it is offering BIP-39 passphrase mode.** This is *not* BIP-38. Knowing which standard you're invoking is non-negotiable for recovery.

Chapter 28

Bitcoin Core descriptors vs BIP-388 wallet_policy

Two interchange formats for the same conceptual object — a wallet's spending rule — coexist in the Bitcoin tool ecosystem. Bitcoin Core ships its own descriptor strings; BIP-388 standardises a JSON-shaped "wallet policy" representation. The m-format md1 card carries a BIP-388 wallet policy; `mnemonic export-wallet` emits either.

This is a density-watch chapter (≤4 pages); detail is intentionally shallow.

28.1 What each format is

	Bitcoin Core descriptor	BIP-388 wallet_policy
Shape	flat string	JSON object
Inline keys	full xpubs in the string	placeholders `@0`, `@1`; xpubs in `keys_info`
Multipath	inline `<0;1>/*`	template-side `@N/**` (per-key path elided into the placeholder)
Master fingerprint	inline `[fp/path]` prefix	in `keys_info` strings
Origin paths	inline	in `keys_info` strings
Hardware-wallet display	full string	template-only (privacy)
Authoritative	Bitcoin Core source	BIP-388 spec
Formal name	"output descriptor"	"wallet policy"

28.2 What the toolkit emits

Both, via `mnemonic export-wallet --format <bitcoin-core | bip388>`:

- **`bitcoin-core` (default)** — JSON array of single-string descriptors, suitable for `bitcoin-cli importdescriptors`. Both receive (`<0;1>/0/*`) and change (`<0;1>/1/*`) descriptors are emitted.

- **`bip388`** — single JSON object with `description_template` (the template string) and `keys_info` (the bound xpubs). Targets hardware-wallet coordinators (Coldcard, Ledger, Foundation Passport) and third-party wallet imports that consume the BIP-388 shape directly. Bitcoin Core's `importdescriptors` RPC does **not** consume this format; use `--format bitcoin-core` for Bitcoin Core.

28.3 Why two formats coexist

Hardware wallets care about privacy: they want to display the *template* of a wallet (e.g., "this is a 2-of-3 sortedmulti") without showing the user a 200-character descriptor. BIP-388 separates the template from its bound keys to make this UI practical.

Bitcoin Core, by contrast, has historically preferred descriptors as a *complete* representation: one string carries everything. The flat string is easy to copy-paste and unambiguous; the trade-off is no separable display.

Bitcoin Core (any version) accepts the descriptor shape via `importdescriptors`. The BIP-388 wallet-policy shape targets hardware wallets and third-party coordinators; Bitcoin Core's `importdescriptors` RPC does not consume it directly (an open issue in Bitcoin Core's tracker proposes adding rendering, but nothing has shipped).

28.4 Side-by-side example

A 2-of-3 sortedmulti, BIP-87 family, three known cosigners:

Bitcoin Core descriptor (single-string form):

```
wsh(sortedmulti(2,
  [fp0/87h/0h/0h]xpub6Cosig0.../<0;1>/*,
  [fp1/87h/0h/0h]xpub6Cosig1.../<0;1>/*,
  [fp2/87h/0h/0h]xpub6Cosig2.../<0;1>/*
))
```

BIP-388 wallet_policy:

```
{
  "name": "wsh-sortedmulti-2-of-3",
  "description": "",
  "description_template": "wsh(sortedmulti(2,@0/**,@1/**,@2/**))",
  "keys_info": [
    "[fp0/87h/0h/0h]xpub6Cosig0...",
    "[fp1/87h/0h/0h]xpub6Cosig1...",
    "[fp2/87h/0h/0h]xpub6Cosig2..."
  ]
}
```

Same wallet; different shapes. The toolkit converts between them via `mnemonic export-wallet`.

28.5 Choosing one format over the other

Receiving software	Use
Bitcoin Core (any version, via `importdescriptors`)	`--format bitcoin-core`
Sparrow	`--format bip388` (sparrow-native export deferred)
Specter	`--format bip388` (specter-native export deferred)
Coldcard / SeedSigner / Foundation Passport	`--format bip388`
Custom tooling	whichever shape is cheapest to parse

(`--format sparrow` and `--format specter` flags are accepted by the binary but currently return a deferral stub; use `--format bip388` for both. A future v0.8.x patch may light up the stubs.)

28.6 What the m-format md1 card carries

md1 stores a BIP-388 wallet-policy *template*: the policy string with `@0`, `@1` placeholders, plus a single bound xpub for the slot it represents. Multi-cosigner reconstruction picks up each cosigner's mk1 and resolves the placeholders at decode time.

BIP-388's separation of template-from-keys was the structural choice that made the m-format possible: each mk1 carries its cosigner's xpub independently, and the shared md1 carries the template. Reconstructing a flat Bitcoin Core descriptor requires mk1 + md1 simultaneously — exactly the cross-binding the `policy_id_stub` enforces.

Chapter 29

Appendix A — Glossary

Definitions are intentionally terse for v0.1; expanded in Phase 7. For deeper background, see the newcomer primers in Appendix B–Appendix E.

29.1 BCH

Bose–Chaudhuri–Hocquenghem error-correction code. The `mk-codec` and `md-codec` use a Bitcoin-tuned BCH polynomial (forked from BIP-93 codex32) to detect and locate engraving errors on each card. The `ms-codec` uses BIP-93 codex32 directly (via `rust-codex32`).

29.2 BIP-32

Hierarchical Deterministic (HD) wallet derivation. Defines extended keys (`xprv` / `xpub`) and child-key derivation paths like `m/84'/0'/0'`. See Appendix C.

29.3 BIP-38

Passphrase encryption of a single private key. Used by the toolkit's `mnemonic convert` subcommand for the `bip38` and `minikey` node types. v0.8 introduced a distinct `--bip38-passphrase` channel separate from the BIP-39 passphrase.

29.4 BIP-39

Mnemonic phrase encoding of wallet entropy as 12, 15, 18, 21, or 24 English (or other-language) words. The "seed phrase" stored on the **ms1** card. See Appendix B.

29.5 BIP-44 / BIP-49 / BIP-84 / BIP-86

Purpose-field constants for single-sig HD wallets: `44'` = legacy P2PKH, `49'` = P2SH-wrapped P2WPKH, `84'` = native SegWit P2WPKH, `86'` = single-key taproot.

29.6 BIP-85

Deterministic child-entropy derivation. From a single master seed, derive deterministic child secrets for other wallets, password strings, raw entropy, or BIP-85 DICE entropy. Exposed via `mnemonic derive-child`.

29.7 BIP-93

The codex32 standard. A Bitcoin-tuned 32-character alphabet with a BCH-style checksum and human-readable prefix. Adopted directly by the **ms1** format; **md1** and **mk1** fork the BCH plumbing.

29.8 BIP-388

Wallet-policy descriptor templates. A canonical JSON shape for exchanging multisig descriptors between wallets. Exposed via `mnemonic export-wallet --format bip388`.

29.9 bundle

The 3-card aggregate (ms1 + mk1 + md1) emitted by `mnemonic bundle`. Each card is independently BCH-checksummed by its sibling codec; the toolkit cross-binds them via the `policy_id_stub`, which is carried on each mk1 card and is computable from each md1 card. (The `mnemonic` toolkit is the integration *layer* over the three card formats; it emits no separate "toolkit card" of its own.)

29.10 card

A single engravable string emitted by one of the three card codecs: **ms1** (secret), **mk1** (key), or **md1** (descriptor). Each card carries its own BCH checksum so partial damage is locatable.

29.11 checksum

The BCH residue suffix on every card. `ms1` uses BIP-93 codex32; `md1` and `mk1` use HRP-mixed BCH (forked from codex32, not upstream-shared).

29.12 codex32

The BIP-93 checksummed alphabet used by `ms-codec`. See **BCH**.

29.13 cosigner

A participant in a multisig wallet, contributing one xpub. The toolkit takes one `--slot @N.<subkey>=<value>` per cosigner when synthesising a multi-source bundle.

29.14 derivation path

A BIP-32 path string like `m/84'/0'/0'/0/0`. Apostrophes mark hardened indices.

29.15 descriptor

A Bitcoin Core wallet description-language string like `wsh(sortedmulti(2,xpub.../84h/0h/0h/0/`. The **md1** card encodes a descriptor.

29.16 DICE

A BIP-85 application emitting deterministic dice rolls (1–6) or n-sided rolls. Useful for paper-friendly child entropy. Added in toolkit v0.8.

29.17 engraving card

A printable layout that includes one card string plus its visual checksum row, formatted for steel engraving. The `--no-engraving-card` flag suppresses generation.

29.18 entropy

Raw byte-level wallet seed material. BIP-39 phrases encode entropy of 128, 160, 192, 224, or 256 bits.

29.19 fingerprint

The first 4 bytes of HASH160 of a public master key. Identifies a wallet origin in BIP-32 / BIP-388 derivations.

29.20 HMAC-SHA-512

The keyed-hash primitive driving BIP-32 child-key derivation and BIP-85 child-entropy derivation.

29.21 m-format constellation

The four sibling formats — **ms1**, **mk1**, **md1**, and the `mnemonic-toolkit` integration layer. The "star" is the visual mental model: toolkit at the centre, three card formats radiating out.

29.22 md1 / md-cli / md-codec

The descriptor card. Encodes a BIP-388-style wallet policy. CLI binary `md`; library crate `md-codec` in repo `bg002h/descriptor-mnemonic`.

29.23 mk1 / mk-cli / mk-codec

The key card. Encodes an xpub plus its BIP-32 origin (master fingerprint + derivation path). CLI binary `mk` (since v0.2); library crate `mk-codec`. Repo `bg002h/mnemonic-key`.

29.24 ms1 / ms-cli / ms-codec

The secret card. Encodes BIP-39 entropy (recovers the seed). CLI binary `ms`; library crate `ms-codec`. Repo `bg002h/mnemonic-secret`.

29.25 mnemonic

The integration CLI binary, shipped by the `mnemonic-toolkit` crate. Five subcommands: `bundle`, `verify-bundle`, `convert`, `export-wallet`, `derive-child`. Multi-source seeds, xpubs, and related wallet inputs flow in via the uniform `--slot @N.<subkey>=<value>` shape (where `@N` is a cosigner index).

29.26 mnemonic phrase

A BIP-39 seed phrase. The plain-text representation of wallet entropy.

29.27 multi

A descriptor key-list constructor: `multi(K, key1, key2, …)` — keys remain in the original order.

29.28 multisig

A wallet that requires *at least K of N* cosigners' signatures to spend. Composed at the script layer, not the seed layer; each cosigner has their own seed. Toolkit support:

`wsh-multi`, `wsh-sortedmulti`, `sh-wsh-multi`, `sh-wsh-sortedmulti`, `tr-multi-a`, `tr-sortedmulti-a`.

29.29 multi_a

The taproot variant of `multi`, defined in BIP-386 (`multi_a` descriptor) and exchanged via BIP-388 (wallet policy).

29.30 NUMS internal key

A Nothing Up My Sleeve elliptic-curve point used as the taproot internal key when a multisig is engraved with no cooperative-spend path. Selected via `mnemonic export-wallet --taproot-internal-key nums`.

29.31 policy_id_stub

A 4-byte stub of `SHA-256(canonical wallet-policy preimage)`, carried on each `mk1` card and computable from each `md1` card. Cross-binds the cards in a bundle so cards from different wallets cannot be mixed.

29.32 SLIP-0132

The convention for prefixing extended-key serialisations with format- identifying bytes (`xpub` / `ypub` / `zpub` / `Ypub` / `Zpub` etc.). Toolkit v0.6 added prefix-tolerant input + a `--xpub-prefix` output flag.

29.33 slot

The toolkit's `--slot @N.<subkey>=<value>` input shape, where `@N` is a cosigner index in a multisig wallet and `<subkey>` is `phrase`, `entropy`, `xpub`, etc.

29.34 sortedmulti

A descriptor key-list constructor that lexicographically sorts keys before script construction; signing is order-independent.

29.35 sortedmulti_a

The taproot variant of `sortedmulti`.

29.36 taproot

BIP-341 single-leaf or multi-leaf P2TR script type. Toolkit supports single-key taproot (BIP-86) and multisig taproot via `tr-multi-a` / `tr-sortedmulti-a` templates.

29.37 threshold (K-of-N)

A multisig parameter: any K of the N cosigners can sign. Set via `--threshold K` (any value 1..=N). Note: K-of-N *secret-share splitting* (splitting the ms1 card itself into N shares) is a separate feature planned for ms-codec v0.2.

29.38 tr (taproot descriptor)

The descriptor outer wrapper for a taproot output: `tr(internal_key, {leaf1, leaf2, …})`.

29.39 verify-bundle

A `mnemonic` subcommand that re-derives expected card content from a seed (or a partial set of cards) and checks parity across the three cards.

29.40 watch-only wallet

A wallet holding only public keys (`xpub`, descriptor) and no signing material. Created via `mnemonic export-wallet`.

29.41 wallet_policy

The BIP-388 JSON shape: `{ name, description, description_template, keys_info }`. Emitted via `mnemonic export-wallet --format bip388`.

29.42 wsh

Witness Script Hash — the descriptor wrapper for native SegWit multisig (`wsh(multi(...))`, `wsh(sortedmulti(...))`).

29.43 xprv / xpub

BIP-32 extended private / public keys. The 78-byte serialisation that propagates across BIP-32 derivations.

Chapter 30

Appendix B — BIP-39 entropy primer

BIP-39 is the standard that turns a randomly-generated chunk of entropy into a sequence of *words*. Almost every Bitcoin wallet software in use today understands BIP-39; the m-format constellation's ms1 card is a checksum-protected encoding of the same entropy.

This appendix is a one-page orientation, not a formal specification. For the spec itself, see BIP-39.

30.1 What BIP-39 does

1. Pick a number of bits of entropy: `128`, `160`, `192`, `224`, or `256`. (The corresponding word counts are `12`, `15`, `18`, `21`, and `24`.)
2. Compute a checksum: the first `bits/32` bits of `SHA-256(entropy)`.
3. Concatenate `entropy || checksum`; the result is a multiple of 11 bits.
4. Slice the bitstream into 11-bit chunks, each indexing into a fixed 2048-word list (`english.txt`).
5. The mnemonic is the resulting word sequence.

To recover the entropy, reverse the slicing and verify the checksum.

30.2 What BIP-39 doesn't do

- **Derive any keys.** That's BIP-32 (see Appendix C). The phrase → 64-byte seed conversion is `PBKDF2(phrase, "mnemonic" || passphrase, 2048, HMAC-SHA-512)`.
- **Specify a wallet template.** That's BIP-44 / 49 / 84 / 86 / 388.
- **Protect against transcription errors.** A 12-word phrase has a 4-bit checksum; a single transcription error has a small chance of producing a *different valid* phrase decoding to a different wallet. The m-format ms1 wraps this with BCH error correction so small stamping errors are detected and located.

- **Per-wallet uniqueness.** A phrase + an empty passphrase is one wallet; the same phrase + a passphrase is a *different* wallet. This is the BIP-39 passphrase / "13th word" convention.

30.3 Wordlist sanity

The 2048-word wordlist is *sorted* and curated to:

- Have unique 4-letter prefixes: the first four letters identify any word in the list unambiguously.
- Avoid pairs that are easily confused (no `won` / `one`, no `their` / `there`).
- Be ASCII-only (with non-English wordlists similarly curated for their respective alphabets).

This makes recovery from partial readability practical: typically the first four letters of each word disambiguate.

30.4 Multi-language wordlists

BIP-39 standardises ten wordlists: English (default), Japanese, Korean, Spanish, Simplified Chinese, Traditional Chinese, French, Italian, Czech, Portuguese. The toolkit accepts all ten via `--language <LANGUAGE>`.

Two wallets created from the same entropy in different wordlists recover to the same wallet — the entropy bits are language-agnostic, the wordlist is just a presentation layer.

30.5 Why ms1 instead of just engraving the BIP-39 phrase?

The m-format ms1 card carries the *entropy* (not the phrase) under a BCH error-correction layer:

Property	BIP-39 phrase on steel	ms1 on steel
Per-character checksum	no	yes (BCH)
Locatable error position	no	yes
Engraving alphabet	English (or other) words; full Roman	32-character codex32 alphabet
Recoverability from N stamping errors	depends on prefix readability	up to a fixed number of bit errors

The phrase remains valuable as a *mnemonic* — humans can read it, hardware wallets accept it directly. The ms1 card's role is the engraved-form-of-record, not a replacement for the phrase.

Chapter 31

Appendix C — BIP-32 derivation primer

BIP-32 is how one secret seed becomes an unlimited tree of public- private keypairs. It is the foundation of every “HD” (hierarchical deterministic) wallet.

For the formal spec see BIP-32.

31.1 The basics

A BIP-32 *master node* is a 32-byte private key plus a 32-byte “chain code” (extra entropy that prevents an attacker who learns one child key from inferring siblings). From any node, two operations produce a child node:

- **Normal (non-hardened) derivation:** `child_i = HMAC-SHA-512(parent_chain_code, parent_pubkey || i)`
- **Hardened derivation:** `child_i' = HMAC-SHA-512(parent_chain_code, 0x00 || parent_privkey || ser32(i))` where the resulting child is labelled with index `i + 2^31`.

The leading `0x00` byte makes the hardened-derivation HMAC input exactly 33 bytes, matching the length of a compressed pubkey used in normal derivation; this is what differentiates the two paths.

Hardened derivation needs the parent’s *private* key; normal derivation can be done with only the parent’s *public* key — which is why xpubs can derive child addresses without exposing privkeys.

31.2 Path notation

A BIP-32 derivation path is a slash-separated sequence:

```
m / 84' / 0' / 0' / 0 / 5
```

- `m` = master.

- ' (or h, or H) = hardened (add 2^31 to the index).
- Subsequent unprimed indices = non-hardened.

The standard wallet path m/84'/0'/0'/0/5 reads: "the 6th external receive address (index 5) of account 0 of Bitcoin (coin 0) with BIP-84 (purpose 84')".

31.3 Standard purpose paths

Purpose	Path	Address shape
BIP-44	m/44'/0'/0'	1... (legacy P2PKH)
BIP-49	m/49'/0'/0'	3... (P2SH-wrapped P2WPKH)
BIP-84	m/84'/0'/0'	bc1q... (native SegWit P2WPKH)
BIP-86	m/86'/0'/0'	bc1p... (single-key taproot)
BIP-87	m/87'/0'/0'	multisig (script-type-agnostic)
BIP-48	m/48'/0'/0'/2'	multisig (script-type indexed via 4th level)

Coin index 0 = Bitcoin mainnet; 1 = testnet/signet/regtest.

The mk1 card carries the *origin* — the master fingerprint plus the hardened path used. Recovery software needs both to know which sub-tree of the master node a given xpub came from.

31.4 Multipath descriptors

Modern wallets use multipath notation <0;1>/* to mean "the same xpub, with chain 0 for receives and chain 1 for change, then any non-hardened child along the wildcard". This is what's actually emitted on md1 cards — a wallet policy with multipath.

```
wpkh([fp/84'/0'/0']xpub.../<0;1>/*)
```

reads "for each chain (0 = receive, 1 = change), all non-hardened children form valid receive/change addresses."

31.5 Why hardening matters for recovery

If an attacker compromises *one* xpub down a non-hardened path, they can derive all sibling xpubs at the same level (and below, if those levels are also non-hardened). Hardening at the *purpose / coin / account* boundary prevents this; child-level non-hardening is acceptable because the children only carry public keys.

The m-format constellation follows this convention: hardened path up to the account level, then non-hardened wildcards for receive/change.

Chapter 32

Appendix D — Descriptors and BIP-388 primer

A *descriptor* is a Bitcoin Core data type that describes a wallet's spending rule completely: the script type, the keys involved, and how to derive specific addresses. *BIP-388* extends descriptors into a wallet-policy format suitable for hardware wallets and multisig coordinators.

For the specs see BIP-380 through BIP-389.

32.1 Descriptor anatomy

A typical descriptor wraps a key (or set of keys) inside one or more *script-context wrappers* that determine how the script is executed:

```
wpkh(  [73c5da0a/84'/0'/0']xpub6Cat.../<0;1>/*  )
```

The pieces:

- `wpkh(...)` — Witness Public Key Hash, the BIP-84 native-segwit outer wrapper.
- `[73c5da0a/84'/0'/0']` — *origin*: master fingerprint + the path used to derive the inner xpub.
- `xpub6Cat...` — the BIP-32 extended public key.
- `/<0;1>/*` — *multipath*: chain 0 for receives, chain 1 for change; * is the wildcard non-hardened address index.

32.2 Common outer wrappers

Wrapper	Address shape	Multisig?
`pkh(...)`	`1...` legacy P2PKH	no
`sh(...)`	`3...` P2SH	yes (legacy multisig)
`wpkh(...)`	`bc1q...` native segwit	no
`wsh(...)`	`bc1q...` (longer) native segwit script	yes

Wrapper	Address shape	Multisig?
`sh(wsh(...))`	`3...` nested segwit	yes
`tr(...)`	`bc1p...` taproot	yes (via tapscript)

Inside multisig wrappers, `multi(K, key1, key2, …)` and `sortedmulti(K, …)` (BIP-67-sorted) construct the K-of-N policy. Taproot uses `multi_a` / `sortedmulti_a` instead.

32.3 What BIP-388 adds

A flat descriptor string is opaque: a hardware wallet can't show the user "this is a 2-of-3 sortedmulti" without showing them the full string. BIP-388 separates the *template* from the *bound keys*:

```
{
  "name": "wsh-sortedmulti-2-of-3",
  "description_template": "wsh(sortedmulti(2,@0/**,@1/**,@2/**))",
  "keys_info": [
    "[fp0/87h/0h/0h]xpub...",
    "[fp1/87h/0h/0h]xpub...",
    "[fp2/87h/0h/0h]xpub..."
  ]
}
```

The template uses placeholders `@0`, `@1`, `@2`; the `keys_info` array binds each placeholder to a concrete xpub at decode time.

The hardware-wallet UI shows the template (verifiable as "2-of-3 sortedmulti") without exposing the per-cosigner xpub strings to a casual viewer.

32.4 Why this matters for the m-format constellation

The md1 card carries a BIP-388 wallet-policy template plus a *single* bound xpub for the slot it represents. Each cosigner's mk1 card carries that cosigner's xpub independently. Reconstructing the flat descriptor requires the md1 + each cosigner's mk1 — the cross-binding the toolkit checks via `policy_id_stub`.

This separation is what enables multi-cosigner air-gapped synthesis: each cosigner produces their own xpub on their own machine; the coordinator builds the wallet-policy template; recovery composes both sides.

32.5 Multipath / <0;1>/* semantics

The `<0;1>/*` notation is BIP-389 (multipath descriptors): one descriptor expresses *both* receive and change paths. Wallet software derives addresses by substituting each value in the multipath set into the position; for `<0;1>/*` that means two distinct address sequences, one for chain 0 (external/receive) and one for chain 1 (internal/change).

The toolkit emits multipath descriptors by default — both chains in one md1 card, no need to engrave receive and change as separate descriptors.

Chapter 33

Appendix E — codex32 / BCH / m-codec error correction

The three card formats (ms1, mk1, md1) all use Bose–Chaudhuri– Hocquenghem (BCH) error-correction codes. ms1 uses BIP-93 codex32 directly; md1 and mk1 fork a Bitcoin-tuned BCH polynomial. This appendix sketches *why* and *how* the codes catch and locate engraving errors.

For the formal specs, see BIP-93 (codex32) and the descriptor-mnemonic / mnemonic-key BIP drafts.

33.1 What a BCH code does

Given a stream of data symbols (each from a 32-character alphabet), a BCH encoder appends a few extra symbols computed by polynomial arithmetic. The decoder, on receiving the stream + checksum, recomputes the polynomial and:

- If no errors: the recomputed checksum matches; data is intact.
- If a few errors: the polynomial syndrome *locates* the error positions. BIP-93 guarantees correction of up to 4 unknown-position substitutions, or up to 8 errors at known positions ("erasures"), or up to 13 consecutive erasures. See *Error detection and correction guarantees per card* below for the precise table.
- If many errors: the polynomial mismatch is uncorrectable; decode fails.

The "few" vs "many" threshold is determined by the polynomial's distance — a property fixed by the polynomial choice, not configurable per-card.

33.2 codex32: BIP-93's contribution

Codex32 specifies:

- An alphabet of 32 characters chosen to avoid visual ambiguity (no `0` / `O`, no `1` / `l`, no `b` / `6` / `8`).
- A BCH polynomial tuned for the codex32 alphabet over GF(32).

- A human-readable prefix mechanism.
- Optional K-of-N share-splitting: one secret splits into N shares, any K reconstruct.

For ms1 (the secret card), codex32's single-string mode is used verbatim via the upstream `rust-codex32` crate. The K-of-N share mode is planned for ms-codec v0.2.

33.3 md / mk: forked BCH plumbing

The md1 and mk1 cards use a *forked* BCH polynomial: same algebraic machinery as codex32, different polynomial constants (tuned for md1's and mk1's payload sizes and HRP-mixing requirements). The fork is an explicit design choice in the descriptor-mnemonic and mnemonic-key BIP drafts; a future `mc-codex32` extraction was considered and *retired* in 2026-05-03 (see the project's CLAUDE.md for the cross-repo coordination record).

33.4 HRP mixing

The md / mk BCH variant mixes the human-readable prefix (`md` or `mk`) into the polynomial computation. Two consequences:

- A card decoded with the wrong HRP fails the BCH check (no risk of mistaking an md1 string for an mk1 string, or vice versa).
- The polynomial constants for md1 and mk1 are distinct, even though the algebra is identical.

Codex32 has its own HRP mechanism in BIP-93; ms1 inherits that.

33.5 Long vs regular code

Each format has two BCH codes:

- **Regular code** — shorter strings, fewer codewords needed, smaller checksum overhead. Used when the payload fits in the regular layout.
- **Long code** — longer strings, larger checksum overhead, used when the payload requires the extended layout (e.g., a multi-cosigner mk1 with many path components).

The toolkit picks the right code automatically based on payload size; no user flag is needed. (`md encode --force-long-code` is accepted on the binary as a forward-compat scaffold but is a documented no-op since md-codec v0.12.0; long-code mode for md1 was dropped on the codec side.)

33.6 Error detection and correction guarantees per card

The numbers below are taken **verbatim from BIP-93** §"Error Correction" and §"Generating the Checksum". They apply to every BCH variant in the constellation that

shares codex32-pattern parameters: the minimum distance is set by the generator polynomial, which is identical across ms1, mk1, and md1; only the target residue differs (HRP-mixed `"mk"` / `"md"` for the forked codes, BIP-93 stock for ms1).

33.6.1 Regular code (n=93 symbols, k=80, 13-symbol checksum)

Used by ms1 (BIP-93 directly), and by mk1 + md1 (forked, same generator polynomial). Applies to data strings of ≤93 bech32 characters.

Property	Guarantee	Source
Detection — characters affected	≥ 1 and ≤ 8 errors guaranteed detected	BIP-93 §“Generating the Checksum”
Detection — random patterns beyond 8 errors	$< 3 \times 10^{-20}$ probability of missed detection	BIP-93 §“Generating the Checksum”
Correction — substitutions (unknown positions)	up to **4** substitutions corrected	BIP-93 §“Error Correction”
Correction — erasures (known positions, e.g., ?)	up to **8** erasures corrected	BIP-93 §“Error Correction”
Correction — consecutive erasures (a single scratch)	up to **13** consecutive erasures corrected	BIP-93 §“Error Correction”

33.6.2 Long code (n=108 symbols, k=93, 15-symbol checksum)

Used by mk1 only (mk1 chunked-mode strings of 96–108 chars). md1 dropped the long code in md-codec v0.11; ms1 v0.1 payloads always fit in the regular bracket.

Property	Guarantee	Source
Detection — characters affected	≥ 1 and ≤ 8 errors guaranteed detected	BIP-93 §“Long codex32 Strings”
Detection — random patterns beyond 8 errors	$< 3 \times 10^{-23}$ probability of missed detection	BIP-93 §“Long codex32 Strings”
Correction (substitutions / erasures / consecutive erasures)	same as regular code: 4 / 8 / 13	BIP-93 §“Error Correction” applied to the longer code

33.6.3 How the code variant is chosen per card

Card	Payload kind (v0.1)	Typical string length	Code variant
ms1	BIP-39 entropy 16–32 B (`entr`)	~70 chars	regular
mk1	xpub + origin (single-string mode)	~52–55 chars	regular

Card	Payload kind (v0.1)	Typical string length	Code variant
mk1	xpub + origin (chunked mode, longer paths)	96–108 chars	long
md1	wallet policy	75–93 chars	regular only (v0.11+)

The toolkit picks the variant automatically based on data length; no user flag is needed. (The "typical string length" column is a best-read estimate pending an empirical sweep across all payload kinds; tracked as `bch-string-length-empirical-sweep` in `docs/manual/FOLLOWUPS.md`.)

33.6.4 One subtlety: HRP mixing does not change the correction guarantees

The forked BCH for mk1 and md1 changes the **target residue** — the constant the polymod is XOR'd against at encode time and compared against at decode time. It does **not** change the **generator polynomial** or the **field arithmetic**, both of which are inherited byte-for-byte from BIP-93. Minimum distance, and therefore the detection / correction guarantees in the tables above, are properties of the generator polynomial alone — so the BIP-93 numbers transfer to mk1 and md1 unchanged. What HRP mixing buys is **format separation**: an mk1 string can never accidentally validate as an md1 or ms1 string, because the target residues disagree.

33.6.5 Errors the BCH code does NOT handle: deletions and insertions

An "erasure" in the table above means *the character at a known position is unreadable, but the position itself is preserved* (typically rendered `?`). The string length is unchanged.

A **deletion** (a missing character that causes everything after it to shift left) is a length-changing event and is **outside the BCH code's correction model**. Same for **insertions** (extra character shifts everything right).

In practice:

- **Detection of length-changing damage is overwhelming.** The decoder first checks that the string length matches one of the expected lengths (≤93 for regular, 96–108 for long). One deletion drops the length by 1; if the encoded length was a known fixed value for the payload kind, the decoder rejects on length mismatch *before* even running the polynomial. Even when a deletion happens to be paired with a compensating insertion (so the length comes out right), ~half the symbols end up in wrong positions and the polynomial syndrome is wildly nonzero. The probability of length-changing damage silently passing the checksum is comparable to the $< 3 \times 10^{-20}$ general missed-detection bound.

- **Correction of length-changing damage is not supported.** The decoder reports "string length out of range" or "checksum invalid" without identifying the missing or extra character's position. The polynomial syndrome encodes substitution-error positions, not insertion / deletion positions.
- **First line of recovery: count the characters on the plate.** Each card has a known string length. If the count is off, recount the engraved plate before attempting to decode — the position of the missing or extra character is something only physical inspection can recover. This is one practical reason the toolkit emits both the contiguous form (handy for copy-paste) and the chunked-in-fives form (handy for engraving) of every string: chunking makes character counts trivially auditable on the plate.

33.7 Hand-decodability with the codex32 volvelle

Codex32 was designed by Leon Olson Curr & Pearlwort Snead as a **paper computer** — a printed rotating-disk volvelle — so that a user can encode, verify, and even substitution-correct codex32 strings without ever touching an electronic device. The BlockstreamResearch/codex32 repo carries the printable volvelle PDFs; the design ethos is also visible in the original BIP-93 draft branch name (`apoelstra/bips/blob/2023-02--volvelles/...`) and the project website secretcodex32.com.

The implication for the m-format constellation is that hand-decodability is a *graded* property across the three cards.

33.7.1 What the wheel does mechanically

A codex32 volvelle physically encodes the BCH polynomial arithmetic. Each input symbol triggers a fixed rotation; the operator reads off the result of the polymod operation step by step. The wheel encodes:

- the **field arithmetic** (GF(32) multiplication via the rotation positions),
- the **generator polynomial coefficients** (the wheel's tooth spacing), and
- an **endpoint comparison** at the last symbol against a fixed target residue. A valid string lands on a marked position; an invalid string lands elsewhere.

The first two are baked into the wheel's physical geometry. The third — the endpoint comparison — depends on which target residue the code uses.

33.7.2 Which constellation cards work directly with the off-the-shelf wheel

Card	Generator polynomial	Target residue	Off-the-shelf wheel?
ms1	BIP-93 stock	`MS32_CONST = 0x10ce0795c2fd1e62a` (BIP-93 stock)	**Yes — directly.** ms1 uses BIP-93 codex32 unchanged via `rust-codex32`. The standard codex32 wheel decodes ms1 strings as-is.
mk1 (regular)	BIP-93 stock	`MK_REGULAR_CONST = 0x1062435f91072fa5` (NUMS-derived, top 65 bits of `SHA-256("shibbolethnums")`)	**Mechanics yes, endpoint no.** Per-step rotations are identical; the final endpoint comparison is against a different 65-bit constant.
mk1 (long)	BIP-93 long-code generator	`MK_LONG_CONST = 0x41890d7e441cbe97273` (NUMS-derived, top 75 bits)	Same as mk1 regular — generator matches BIP-93 long, endpoint differs.
md1 (regular)	BIP-93 stock	`MD_REGULAR_CONST = 0x0815c07747a3392e7` (NUMS-derived, top 65 bits of `SHA-256("shibbolethnums")`)	**Mechanics yes, endpoint no.** Same as mk1 regular.

A user who wants to verify an ms1 card on the kitchen table with a printed codex32 volvelle can do so directly. A user who wants to verify mk1 or md1 with the same wheel needs either:

1. **A printed target-residue card.** Carry a small print-out listing all three target residues; do the final XOR-against-target step manually after the wheel computes the polymod. Straightforward but reduces the elegance of the volvelle's all-on-the-wheel property.
2. **An mk1- or md1-specific volvelle** with endpoint marks shifted to match the HRP-mixed target residue. First-cut v0.1 wheels for the m-format constellation NUMS-derived BCH residues are now available in this repo: mk1-regular wheel, mk1-long wheel, and md1-regular wheel. Hand-decoding ergonomics will improve in v0.2 — see `bottom-disc-cell-density` and `bottom-disc-registration-tick-radius` in `docs/manual/FOLLOWUPS.md`.

33.7.3 Why HRP mixing was accepted at this cost

Format separation. An mk1 string cannot accidentally validate as an ms1 or md1 string at decode time, because a wallet decoding mk1 compares the polymod against `MK_REGULAR_CONST` / `MK_LONG_CONST` while a wallet decoding ms1 compares against `MS32_CONST`. The probability that random bit-flipping turns a valid ms1 string into a valid mk1 string is the probability of the polymod landing on `MK_REGULAR_CONST` instead of `MS32_CONST` — combinatorially implausible with NUMS-derived constants.

The trade-off: stock-volvelle compatibility for mk1 / md1 was given up in exchange for cryptographically strong format separation. ms1 retains stock-volvelle compatibility because it uses BIP-93 directly. For a user who specifically wants hand-recovery without electronics, the practical recipe today is:

- Print the BlockstreamResearch volvelle for ms1 verification.
- Print the v0.1 HRP-specific wheels for mk1 and md1 verification: mk1-regular wheel, mk1-long wheel, md1-regular wheel.
- Or carry a small printed card listing the three target residues (`MS32_CONST`, `MK_REGULAR_CONST`, `MD_REGULAR_CONST`); do the final XOR step by hand for mk1 / md1.

33.7.4 Sources for further reading

- **BIP-93** — the canonical codex32 specification, including the §"Error Correction" claims quoted above and a Python reference implementation of the polynomial.
- **BlockstreamResearch/codex32** — the original codex32 paper-computer work by Leon Olson Curr & Pearlwort Snead that became BIP-93.
- **apoelstra/rust-codex32** — Andrew Poelstra's CC0 Rust reference implementation. ms-codec depends on `codex32 = "=0.1.0"` directly via this crate.
- mk1's forked BCH plumbing: `crates/mk-codec/src/string_layer/bch.rs` in the mnemonic-key repo.
- md1's forked BCH plumbing: `crates/md-codec/src/bch.rs` in the descriptor-mnemonic repo.
- The `mc-codex32` shared-crate extraction (which would have unified the mk1/md1 fork with rust-codex32) was considered and retired on 2026-05-03; see CLAUDE.md in any of the four repos for the coordination record.

33.8 Operational fallbacks

When stamping errors exceed the per-card correction radius (4 substitutions or 8 erasures in known positions), the fall-backs are:

- **Single-sig:** re-derive the lost card from the seed phrase plus the wallet template — `mk1` and `md1` are public material and are reconstructable from `ms1` plus knowledge of the template (chapter 35 walks through every scenario).
- **Multisig:** for a 2-of-3 wallet, any one cosigner's `ms1` may be lost without losing spending capability (the threshold absorbs it); two cosigner `ms1` losses are fatal.

When length-changing damage (deletion / insertion) is suspected but the character

count looks right, the operator should manually re-decode each five-character chunk against the original digital bundle to spot the mis-aligned chunk.

Chapter 34

Appendix F — Test seeds and example data

Every worked example in this manual uses public test seeds. The canonical example is the BIP-39 12-word *all-zeros* seed:

```
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
↪  about
```

This is the standard test vector from the BIP-39 specification. *Anyone* with a Bitcoin wallet that supports BIP-39 can reproduce the addresses derived from it.

> The phrase `abandon abandon abandon … about` — and any other phrase in this manual — is **public**. Chain watchers continuously sweep the addresses derived from these seeds; any funds sent to them are stolen near-instantly. **Never engrave or fund a wallet built from a test seed.** Generate fresh entropy on an air-gapped device before doing any real work.

34.1 Why a public test seed

Test seeds make the manual *reproducible*. A reader who runs `mnemonic bundle --slot @0.phrase="abandon abandon … about"` gets the same ms1, mk1, md1 strings printed in the manual. Mismatches surface as a bug or a manual-version drift, not as "different seeds."

Production seeds, by contrast:

- Should be generated on an air-gapped machine.
- Should never appear in a manual, a script, a chat log, or a screenshot.
- Should be derived from hardware sources (dice rolls, hardware RNGs in dedicated devices) — software RNGs on networked machines have a long history of compromise.

34.2 Other test vectors used in this manual

For multisig examples, the manual uses two other canonical BIP-39 test vectors:

```
legal winner thank year wave sausage worth useful legal winner thank yellow
letter advice cage absurd amount doctor acoustic avoid letter advice cage above
```

Both are public, both are swept. They are used for cosigner-1 and cosigner-2 in the multisig walkthroughs (chapter 32) and the taproot-multisig walkthrough (chapter 33).

34.3 Network-aware test data

Test seeds are network-agnostic — the same entropy produces addresses on mainnet, testnet, signet, and regtest. The toolkit takes `--network <NETWORK>` separately; the manual's worked examples mostly use `--network mainnet` for the addresses to match what production tooling expects, but readers can substitute testnet for any of them safely.

34.4 Reading test transcripts

The `docs/manual/transcripts/` directory carries pinned `.out` transcripts of the worked examples, one per example. Each transcript was captured against the toolkit binary version named in the chapter (`mnemonic 0.8.0` for the v0.1 of the manual). The `make verify-examples` target re-runs each command and diffs against its transcript; drift is a CI failure.

Updates to a worked example require updating both the chapter prose and the matching transcript in lockstep.

Chapter 35

Appendix G — Troubleshooting matrix

Common failure modes when building, verifying, or recovering an m-format constellation bundle, ordered by frequency.

35.1 Bundle synthesis fails

Symptom	Likely cause	Fix
`error: a value is required for '--template <TEMPLATE>'`	Forgot `--template` (or `--descriptor`)	Pick one of `bip44`/`bip49`/`bip84`/`bip86`/`wsh-multi`/`wsh-sortedmulti`/`sh-wsh-multi`/`sh-wsh-sortedmulti`/`tr-multi-a`/`tr-sortedmulti-a`, or supply `--descriptor`.
`error: a value is required for '--threshold <THRESHOLD>'`	Multisig template without `--threshold K`	Pass `--threshold K` (1 ≤ K ≤ N).
`error: invalid mnemonic`	Phrase typo, wordlist mismatch, or non-canonical wordlist	Check word spellings; pass `--language <LANGUAGE>` if the phrase isn't English.
Bundle synthesis silently produces a different xpub from the wallet you intended	Wrong `--template`, `--account`, or `--network`	Run `mnemonic convert --from phrase=… --to xpub --template <T>` and compare to your wallet's xpub *before* stamping.

35.2 verify-bundle fails

Symptom	Likely cause	Fix
`ms1_decode: error at position N`	Mis-stamped or mis-typed character at position N	Inspect that character against the original digital bundle; correct.
`mk1_xpub_match: mismatch`	Wrong cosigner xpub bound to the slot, or wrong template/account/network	Re-derive the expected xpub from the slot's seed; if it differs from the mk1's xpub, the mk1 was bound for a different cosigner.
`md1_xpub_match: mismatch`	md1 carries a different cosigner's xpub for the slot than the multisig set expects	Verify each cosigner's slot index; the md1 binds slot `@N` to a specific xpub and the verifier compares to the slot input.
`policy_id_stub: mismatch`	Cards from different bundles mixed	Confirm all cards came from a single `mnemonic bundle` invocation.

35.3 convert / recovery

Symptom	Likely cause	Fix
`error: BIP-38 decryption failed`	Wrong `--bip38-passphrase` or wrong source key	Try the alternative passphrase channel (v0.7 used a single `--passphrase`; v0.8 split).
Empty stdout from `convert --to phrase`	Source node was an xpub (no privkey to derive phrase from)	Phrases require entropy or seed; xpub is public-only.
Wrong-network xpub prefix on output	`--network` mis-set or `--xpub-prefix` mis-set	Pass the matching network; for SLIP-0132 prefixes (zpub, ypub, …) use `--xpub-prefix`.

35.4 Engraving / stamping

Symptom	Likely cause	Fix
Verifier passes on digital bundle but fails on engraved cards	Stamping error (e.g., `0` vs `o`)	Re-decode the engraved card; the BCH error position narrows to the offending character.
Plate has a single ambiguous character	Glance bias (e.g., `s` could be `5`)	Use a magnifier; the codex32 / m-format alphabets exclude visually-similar characters but stamping artefacts can still confuse.
Plate damaged beyond the BCH correction radius	Physical destruction beyond the codec's repair limit	Re-derive the card from the seed and re-stamp a fresh plate.

35.5 Wallet-export

Symptom	Likely cause	Fix
Bitcoin Core returns "non-canonical descriptor"	Old Bitcoin Core version	Pass `--bitcoin-core-version 24` for Bitcoin Core 24, or upgrade.
Sparrow/Specter rejects the file	`--format sparrow` / `--format specter` are accepted by the binary but currently return a deferral stub	Use `--format bip388` (or `--format bitcoin-core`); import the resulting JSON via the wallet's BIP-388 / descriptor-import dialog.
Address scan misses recent receives	`--range` too small	Pass a larger `--range`, or re-scan the chain via `bitcoin-cli rescanblockchain`.

35.6 BIP-85 derive-child

Symptom	Likely cause	Fix
`application not in scope` for `rsa` / `rsa-gpg`	These are deferred to v0.9 (RUSTSEC-2023-0071)	Use a different application or wait for v0.9.
`--length 0 invalid for application bip39`	`bip39` requires `--length` 12, 18, or 24	Use a valid word count.
`--dice-sides required for application dice`	DICE app needs the side count	Pass `--dice-sides N` (2..=2^32-1).

35.7 When in doubt

1. **Re-read the chapter.** Most failures trace to a small flag-set discrepancy between the example and the actual command.
2. **Run `--help` directly.** The cli-help snapshots in this manual match toolkit v0.8.0; later versions may have added flags.
3. **Run `verify-bundle` on the engraved cards.** It is the single most useful diagnostic.
4. **Compare against the canonical test seed.** If a worked example from the manual doesn't reproduce, the toolkit version may have drifted; check the manual's release-history appendix (Appendix H) for which toolkit version this manual was built against.

Chapter 36

Appendix H — Release history

This appendix is hand-curated for v0.1 of the manual; subsequent versions will switch to auto-extraction from each repo's `CHANGELOG.md` (filed as `release-history-auto-extract` in `docs/manual/FOLLOWUPS.md`).

36.1 mnemonic-toolkit

Version	Date	Highlights
`0.8.0`	2026-05-07	**BREAKING** composite-edge BIP-38 passphrase split (`--passphrase` for BIP-39, `--bip38-passphrase` for BIP-38). `--passphrase-stdin` for the BIP-38 V3 NULL-byte case. Four non-English Electrum wordlists. `--taproot-internal-key` (`nums` or `@N`) on `export-wallet`. `--descriptor` + `--format bip388` interop. BIP-85 DICE shipped.
`0.7.1`	2026-05-07	Multi-repo BIP test-vector audit-cycle. ~43 net-new pinned vectors. Minor SPEC erratum corrections.

Version	Date	Highlights
`0.7.0`	2026-05-06	New subcommands: `mnemonic export-wallet` (Bitcoin Core, BIP-388 stub, Sparrow / Specter stubs) and `mnemonic derive-child` (BIP-85 entropy / hd-seed / xprv / hex / password-base64 / password-base85). 4 new `convert` node types.
`0.6.x`	2026-05-06	`mnemonic convert` subcommand; BIP-39 / BIP-38 / WIF / xpriv edges; SLIP-0132 prefix-tolerant input.
`0.5.x`	2026-05-06	Unified `--slot @N.<subkey>=<value>` input shape. Legacy CLI flag retirement.
`0.4.x`	2026-05-05	BIP-388 + multi-leaf taproot + schema-4.
`0.3.x`	(early)	Descriptor-mode foundation.
`0.2.x`	(early)	Multisig foundation.
`0.1.x`	(early)	Single-sig foundation.

36.2 descriptor-mnemonic / md-codec / md-cli

The md1 format. Tracking notable releases:

Version	Date	Highlights
`md-codec 0.16.2`	2026-05-07	v0.7.1 audit-cycle close-out; BIP-388 388.2 test vector pinned.
`md-codec 0.16.0`	2026-05-03	CLI extracted to `md-cli` crate; library-only `md-codec`.
`md-codec 0.11.x`	2026-05	Wire-format cleanup; path dictionaries dropped.
`md-codec 0.10.x`	2026-04-29	`OriginPaths = 0x36`; per-`@N` divergent-path encoding.

Version	Date	Highlights
`md-codec 0.9.x`	2026-04-29	BIP-submission gate cleared; `chunk_set_id` rename.

36.3 mnemonic-key / mk-codec / mk-cli

The mk1 format. Standalone CLI `mk` shipped in v0.2 alongside `mk-codec` v0.2.

Version	Date	Highlights
`mk-cli 0.2.0`	2026-05-08	Standalone CLI with `encode`, `decode`, `inspect`, `verify`, `vectors` subcommands; `--from-md1` cross-repo policy-id-stub derivation.
`mk-codec 0.2.2`	2026-05-07	Path-dictionary mirror retirement (post md-codec v0.11). v0.7.1 audit-cycle close-out.
`mk-codec 0.2.0`	2026-04-30	Stable encoder/decoder surface; path-dictionary v1.
`mk-codec 0.1.0`	2026-04-29	Initial release, v0.1 wire format.

36.4 mnemonic-secret / ms-codec / ms-cli

The ms1 format. BIP-93 codex32 directly via `rust-codex32`.

Version	Date	Highlights
`ms-codec 0.1.0` + `ms-cli 0.1.0`	2026-05-03 .. 04	Initial release: BIP-93 codex32 single-string mode. K-of-N share splitting deferred to v0.2.
`ms-codec 0.1.1`	2026-05-07	v0.7.1 audit-cycle: extra corpus vectors; BIP-93 §“Test vectors” cross-format pinning.

36.5 Cross-repo coordination

The four-format star coordinates via the mirror-invariant protocol documented in `descriptor-mnemonic/CLAUDE.md`: any cross-format work surfaces in the originating repo's `design/FOLLOWUPS.md` with a primary entry, and lands a companion entry in each affected sibling. The retired path-dictionary mirror (md-codec v0.11) was the last invariant of this kind to close in lockstep across mk1.

For the in-progress and shipped milestones across the four repos, see each repo's `CHANGELOG.md` directly; the manual is a snapshot as of v0.1's tag.

Chapter 37

Appendix I — Index of terms

The PDF render emits a true page-numbered alphabetical index (built by `makeindex` from `\index{}` markers throughout the source). The markdown render emits this curated `Term → §section` table instead, since markdown viewers have no notion of page numbers.

The two indexes are kept in lockstep by the bidirectional consistency check in `tests/lint.sh`: every `\index{TERM}` marker in the source must have a matching row here, and vice versa. Adding a marker without adding the row (or vice versa) fails the lint.

Term	Section
BCH	Welcome to the m-format constellation
BCH error correction	Single-sig steel-engraved backup
BIP-32	Concept signposts
BIP-39	Concept signposts
codex32	Concept signposts
cross-binding	Single-sig steel-engraved backup
descriptor	Concept signposts
HMAC-SHA-512	Deterministic child secrets via BIP-85
m-format constellation	About this manual
md1	Welcome to the m-format constellation
mk1	Welcome to the m-format constellation
mnemonic-toolkit	Welcome to the m-format constellation
ms1	Welcome to the m-format constellation
multisig	Concept signposts
policy_id_stub	Welcome to the m-format constellation
slot	Concept signposts

Chapter 38

Build banner

Built from commit `e0a2071` on `2026-05-09T01:27:25Z` for toolkit `volvelle-v0.1.3`.

This page is included in the PDF render only.

Index